¡Perfecto! Montemos un repo global que opere como tu “Skill Fabric” y que se conecte a MemTech para minar todos tus repos, generar skills desde ADRs y servirlos a cualquier proyecto (CLOOP/BMCC/Zen-Hub). Aquí tienes el plan ejecutable.

1) Nuevo repositorio (global)

Nombre sugerido: skill-fabric (neutro).

```
skill-fabric/
├─ packages/
│  ├─ skills-cli/              # CLI: init/lint/pack/install/list/run/mine
│  ├─ miner/                   # Minado multi-repo (ADRs + patrones de código)
│  ├─ curator/                 # Filtra, de-duplica, clasifica (guideline/guardrail/…)
│  ├─ builder/                 # Materializa SKILL.md + resources + scripts
│  ├─ evaluator/               # Ejecuta escenarios, smoke y KPIs
│  ├─ writer/                  # Crea/actualiza ADRs de adopción
│  ├─ router/                  # Hooks del editor (pre-invoke/stop)
│  ├─ sandbox-runner/          # Ejecuta scripts de skill aislados (sin red)
│  ├─ memtech-bridge/          # Conector L1-L3 (index, admission, snapshots)
│  ├─ mcp-adapters/            # fs, git, pm2, metrics (Zen Hub MCP)
│  └─ kpi/                     # Emite eventos JSONL/Prom para BMCC/S-Framework
├─ skills/                     # Biblioteca canónica (fuente)
│  ├─ guidelines/…             # estilo/capítulo por dominio
│  ├─ guardrails/…             # bloqueos/verificaciones
│  ├─ workflows/…              # procedimientos (CLOOP)
│  ├─ analysts/…               # auditorías, mapa repo, PR reviewer, etc.
│  └─ generators/…             # plan-architect, testing-plan-designer, etc.
├─ registry/                   # Índices compilados para carga rápida (metadatos)
│  ├─ index.json               # front-matter de todos los skills (para router)
│  └─ bundles/…                # paquetes listos (on-demand)
├─ configs/
│  ├─ skill-rules.schema.json  # esquema (router)
│  ├─ SKILL.template.md        # plantilla ≤400 líneas (progresivo)
│  └─ repos.yaml               # lista de repos a minar (paths/tags/políticas)
├─ scripts/pm2/ecosystem.config.cjs
├─ obs/kpi/events.jsonl        # eventos de desempeño (JSON por iteración)
└─ docs/                       # guía de uso, contratos, ejemplos
```

Variables de entorno (globales)

- SKILLS_HOME=/ruta/a/skill-fabric/registry
- MEMTECH_URL=..., MEMTECH_TOKEN=...
- ZEN_HUB_URL=... (si aplica), GIT_* (token/ssh)

2) Instalación y uso básico

git clone <tu-remote>/skill-fabric && cd skill-fabric
pnpm i -w
pnpm -w build
pnpm -w link --global skills-cli # Exponer CLI global “skills”

En tus proyectos consumidores (cualquier repo):

- Exporta SKILLS_HOME en el perfil de la IDE/terminal.
- Copia los hooks del editor desde packages/router o consúmelos vía MCP.

  El pre-hook leerá registry/index.json; el stop-hook hará prettier→typecheck→hints→KPIs.

3) Conexión con MemTech (bridge)

- L2 (índice): registry/index.json ↔ MemTech Index (metadatos: name, description, tags, type, version, resources).
- L3 (cuerpo): skills/<dominio>/<skill>/SKILL.md y resources/ ↔ Documento completo en MemTech (admission + hash).
- Operaciones expuestas por memtech-bridge:
  - index.upsert(skill_meta), doc.upsert(skill_body), snapshot.create({task, plan}), link.adr(skill_id, adr_id).

4) Minado multi-repo (ADRs + patrones)

Configura configs/repos.yaml:

```
roots:
  - path: /dev/proyectos/app-uno
    tags: [frontend, api, prisma]
  - path: /dev/proyectos/app-dos
    tags: [etl, airflow]
rules:
  adr_globs: ["docs/adr/**/*.md", "ADR/**/*.md"]
  code_globs:
    - "src/**/*.ts"
```

```
  - "services/**/controllers/**/*.ts"
ignore: ["**/node_modules/**", "**/dist/**"]
```

Ejecuta:

```
skills mine --repos configs/repos.yaml --out ./.tmp/candidates.json
skills curate ./.tmp/candidates.json --out ./.tmp/curated.json
skills build  ./.tmp/curated.json --to ./skills
skills eval   ./skills --report ./.tmp/eval.json
skills index  ./skills --out ./registry/index.json
skills publish --to memtech          # sube L2/L3
```

Qué hace el miner:

- ADR-Miner: extrae reglas, DoD, anti-patrones, checklists → candidates.
- Pattern-Miner (código): busca “smells” y convenciones (controlador→servicio→repo, Prisma, routers UI, testing gaps).
- Dedup/Curator: combina señales de ADR y patrón en código, clasifica en guideline|guardrail|workflow|analyst|generator, propone description única (clave para activación), y severidad (suggest|require|block).
- Builder: materializa SKILL.md + resources/ + scripts/ (progresivo, sin simulaciones).
- Evaluator: corre 2–3 escenarios reales por skill en sandbox, emite KPIs.

5) Router/Hooks del editor (global)

- Pre-invoke: lee SKILLS_HOME/index.json, calcula match por intención (keywords/regex), path y contenido; inyecta nota “Skill Activation Check” y carga solo metadatos; si match fuerte, carga el SKILL.md (cuerpo) y, si hace falta, recursos.
- Stop-hook:
    1. Prettier sobre archivos editados
    2. Type-check/build del repo afectado
    3. Hints de manejo de errores (controladores/DB)
    4. Auto-resolver opcional si ≥N errores
    5. KPIs → obs/kpi/events.jsonl (tokens/op, latencia, adherencia, cero-errores, etc.)

6) Integración con CLOOP/BMCC/Zen-Hub/MCP

- CLOOP (Planning duro):
    - skills run plan-architect --task "..." → genera plan.
    - skills run plan-save-workflow --task "..." → crea /dev/active/<tarea>/{plan,context,tasks}.md + snapshot MemTech.
    - Pre-hook bloquea ejecución si no hay plan aprobado.
- Zen-Hub MCP: mcp-adapters/ exponen fs/git/pm2/metrics como tools.
- BMCC/S-Framework: kpi/ exporta métricas (JSONL/Prom).
- Memoria (MemTech): snapshot de planes/skills adoptados, evidencia y enlaces a ADRs.

7) Biblioteca mínima (equivalentes “coder-rabbit-like”, local)

Workflows/Generators

- plan-architect — plan CLOOP detallado.
- plan-save-workflow — tríada + snapshot MemTech.
- testing-plan-designer — matriz mínima de pruebas y comandos.
- pm2-monitor — logs/restart/monit de servicios.

Guidelines

- frontend-dev-guidelines — layout, estado, accesibilidad, Query/Router.
- backend-dev-guidelines — controladores→servicios→repos→DTOs→errores.
- api-contracts-guidelines — OpenAPI, versionado, compatibilidad.

Guardrails

- database-verification (block) — prohíbe queries peligrosas (sin where/select, deleteMany sin confirmación), frena migraciones destructivas sin rollback.
- secrets-and-config (require) — evita secretos en código, valida .env.example.
- migration-safety (require).

Analysts (productividad)

- repo-auditor — mapa del repo, hotspots, deuda técnica, plan de refactor.
- pr-reviewer — resumen/impacto/checklist con test scaffolding.
- test-scaffolder — genera esqueletos de pruebas unit/e2e.
- error-fixer — corrige errores TS/linters frecuentes.
- dep-risk-advisor — licencias/CVEs/actualizaciones.
- dead-code-pruner — detecta módulos huérfanos y propone PR.

> Todos con scripts reales y ejecución en sandbox (sin red por defecto).

8) Distribución a proyectos

Opción A (auto-carga por router): el pre-hook usa SKILLS_HOME → carga metadatos/bundles sin copiar nada al repo consumidor.

Opción B (instalar en repo):

```
cd /ruta/proyecto
skills install backend-dev-guidelines      # copia skill y actualiza router local
```

Opción C (submódulo/paquete): exponer registry/ como paquete interno; consumir vía PNPM workspace o submódulo git (solo si necesitas versionar por proyecto).

9) PM2/Operación (global)

skill-fabric puede correr miner/curator/evaluator en segundo plano:

```
pm2 start scripts/pm2/ecosystem.config.cjs
# incluye: miner (cron), curator, indexer, kpi-exporter
```

- Cron minado (ej. cada noche): vuelve a generar candidatos, evalúa y actualiza el índice.

- Logrotate y max_memory_restart para estabilidad.

10) KPIs y gates

Evento por iteración (obs/kpi/events.jsonl):

```
{
  "ts":"2025-10-29T12:00:01Z",
  "repo":"app-uno",
  "task":"feature-x",
  "skills":["backend-dev-guidelines","database-verification"],
  "adherence":true,
  "errors_ts":1,
  "latency_ms":8200,
  "tokens_total":24500,
  "zero_errors_left_behind":true
}
```

Gates en CI (del repo global o consumidor):

- skills lint (descripciones únicas/claras, sin solapes).

- skills eval (escenarios PASS).

- no-mess-left-behind (post-hook PASS: 0 errores TS, linters OK).

- tokens_per_operation y mean_fix_latency bajo umbral.

11) Roadmap de 7 días (en foco)

Día 1–2: Bootstrap repo + CLI básico + hooks + plantilla SKILL.

Día 3: MemTech-bridge + registry/index.json + instalación de hooks en tu IDE.

Día 4: Miner (ADR + patrones) + Curator + Builder → 5 skills base.

Día 5: Evaluator + KPIs + PM2 (cron minado nocturno).

Día 6: Guardrails DB + pm2-monitor + testing-plan.

Día 7: Integración en 2–3 repos reales (happy paths A/B/C), ADRs de adopción y primeros informes BMCC.

Si quieres, te entrego ahora mismo el starter pack pegable (plantillas de SKILL.md, skill-rules.schema.json, repos.yaml, ecosystem.config.cjs, hooks y un skills-cli mínimo con comandos init|lint|mine|build|index|publish).

¡Sí, y conviene hacerlo ya! Te dejo un análisis ultra-práctico del informe y, a partir de eso, cómo aterrizarlo en tu ecosistema (CLOOP + BMCC + Zen Hub + MemTech) para que los agentes ADR generen y usen skills de forma rutinaria, sin “saltar” pasos en Cursor.

Lo valioso del informe (qué copiamos tal cual)

- Archivo de configuración persistente del agente en el repo que enumera comandos, rutas clave, directrices de estilo y pruebas. Es la “fuente de verdad” que ancla el comportamiento del agente en cada proyecto.

- Flujo “Explorar → Planificar → Codificar → Confirmar” (siempre plan antes de código, y validación explícita al final). Esto encaja perfecto con CLOOP.

- Anatomía de un skill: directorio con SKILL.md (front-matter name + description) y recursos opcionales; el modelo decide relevancia por la description (primacía absoluta).

- Divulgación progresiva (3 niveles): solo metadatos siempre visibles; cuerpo del SKILL.md se carga on-demand; recursos adicionales también on-demand. Ahorra tokens y escala.

- Ejecución en sandbox y orquestación por shell (leer archivos/ejecutar scripts; solo stdout/stderr entra al contexto). Seguridad y eficiencia.

- Cuándo usar “tools” vs “skills”: tools = llamada discreta y síncrona; skills = guiar workflows multi-paso e inyectar procedimiento. Útil para tu router de decisiones.

Dónde podemos mejorar (tu ventaja)

1. Renombrar patrones y rutas (neutral, sin marcas)
    - Introduce AGENT.md (equivalente al doc persistente del informe) en raíz de cada repo; no uses nombres de empresas. Contiene: comandos, rutas,

linters, pruebas, “switches” de CLOOP (C/L/O/O/R) y gates BMCC. (Basado en la pauta del informe).

2. Skill Registry versionado por MemTech
    - Indexa el front-matter (name, description, tags, scope) en MemTech-L2; guarda SKILL.md + plantillas en L3. Activa divulgación progresiva leyendo solo metadatos y cargando cuerpos on-demand (tu router ya lo soporta).
3. ADR→Skill (pipeline automático)
    - Los agentes ADR extraen patrones de ADR (pre-sprints, auditorías, checklists, test-scaffolding) y generan skills con SKILL.md estandarizado + recursos (reference.md, scripts).
    - Valida description (clave para descubribilidad) y colapsa cualquier when_to_use dentro de description (el informe observa que ese campo no está soportado/estable).
4. Hooks + Guardrails by design
    - Pre-hook: Selección de skills por patrones de intención y políticas (OPA/Reglas YAML→Rego).
    - Post-hook: compilación/lint/tests automáticos; si falla, replanificar (CLOOP → volver a “Plan”). (El informe enfatiza validar al final del ciclo).
5. Skill taxonomy para tus casos reales (local-first)
    - Foundational: repo-map, test-scaffold, build-checker, refactor-plan, doc-router.
    - Governance: BMCC gates (lint epistemic), claim-trace, policy-check (OPA).
    - Productividad (sustituye herramientas de pago): checkpoint planner, pre-sprint generator, audit-runner, PR-reviewer con tests, bug-to-fix loop (sandbox).
    - Cada skill en sandbox y sin red por defecto; opt-in explícito si requiere IO.
6. KPIs de adopción y coste (lo de “progresiva” ahorra tokens; mídelo)
    - Tokens/op, latencia media, tasa de replanificación, MTTD/MTTR, “zero-errors-left-behind”. (Encaja con tu BMCC y telemetría).

Plan de implementación (mínimo viable, en tu sistema)

A. Estructura y estándares (1 día)

- /.agent/AGENT.md (plantilla CLOOP) + /.agent/skills/ (biblioteca local).
- Convención de skill:

```
skills/<domain>/<skill-id>/
  SKILL.md      # front-matter YAML + cuerpo
  reference.md  # opcional
  scripts/*     # opcional (se ejecutan en sandbox)
  tests/*       # opcional (smoke)
```

- (Anatomía y metadatos basados en el informe).

B. Tooling CLI (2–3 días)

- cloop-skills init|lint|pack|install|list|run
    - init: genera esqueleto con front-matter correcto.
    - lint: valida description (longitud, verbos de acción, anti-solape). (La description es la señal principal de activación; dale cariño).
    - pack: crea bundle local (sin red).
    - run: ejecuta en sandbox y solo pasa stdout/stderr al agente.

C. ADR→Skill Builder (1–2 días)

- Agente ADR “Curator” + “Generator”:
    1. Detecta patrones en ADRs (pre-sprint, auditoría, checkpoints).
    2. Emite SKILL.md + reference.md + scripts/.
    3. Corre cloop-skills lint y registra en MemTech (L2/L3).
    4. Abre PR con el nuevo skill y tests.

D. Hooks y selección (1 día)

- Pre-hook de intención (router): si coincide con description → cargar solo metadatos y decidir. Si “match fuerte”, cargar SKILL.md y luego recursos según referencias (progresivo).

- Post-hook: compilar/lint/test; si falla → CLOOP vuelve a “Plan”.

E. Zen Hub + PM2 (0.5 día)

- Procesos: memtech, zen-hub, cloop-router, sandbox-runner, skill-registry (con autorestart y logrotate).

- KPIs a Prom/Logs: tokens/op, replan rate, pass rate.

F. Happy paths (operación rutinaria en Cursor)

1. Crear plan (CLOOP-Plan) → router sugiere skills candidatos por description.
2. Cargar skill (progresivo) → SKILL.md trae el procedimiento y checklists.
3. Ejecutar en sandbox → scripts generan artefactos; stdout al contexto.
4. Confirmar → post-hook ejecuta pruebas y valida gates BMCC → merge/PR.

Backlog incremental (rápido → avanzado)

1. MVP: 5 skills foundational + ADR→Skill Builder + hooks pre/post + sandbox.
2. Governance: policies OPA para activación/uso y “no red por defecto”.
3. Quality: smoke tests por skill; matriz de solape para evitar ambigüedad en description.
4. Productividad: pack de skills “Coder-Rabbit-like” (checkpoints, pre-sprints, auditorías, PR-review con test-gen).
5. Auto-mejora: métrica “tokens desperdiciados por carga irrelevante” y reglas para reducirla (beneficio directo de la divulgación progresiva).

Plantilla sugerida de

SKILL.md

(resumen)

---
name: audit-runner
description: Ejecuta auditorías de código y documentación en repos Node/TS, prioriza fallos críticos, propone fixes y reintenta hasta pasar linters y tests mínimos.
tags: [audit, tests, lint, governance]
inputs:
  - repo_path
  - scope: [api, ui, infra]
risks:
  - no_network: true
  - fs_access: read-only

Procedimiento
1) Mapear repo (si no hay index, crear).
2) Ejecutar linters/format.
3) Generar pruebas mínimas si faltan.
4) Proponer/ejecutar fixes en sandbox.
5) Registrar reporte y “ready-to-merge”.
Recursos
- reference.md
- scripts/run_audit.sh

¿Próximo paso? Te genero ahora el esqueleto de la biblioteca (/ .agent/skills/…) con 5 skills base + el CLI cloop-skills (stub) y el hook de selección progresiva para que lo pegues en tu repo y empieces a iterar. Si quieres, dime el nombre del repo y lo adapto a tu estructura de carpetas.

¡Perfecto! Aquí tienes el plan detallado y ejecutable para montar, dentro de tu ecosistema, un entorno mínimo que soporte una biblioteca de skills y su uso rutinario con agentes ADR, CLOOP, BMCC, Zen-Hub (MCP), MemTech, guardrails, PM2 y KPIs —todo local y sin depender de herramientas de pago.

---

1) Objetivo y alcance

Meta: Dejar operativo un “Skill Fabric” local que:

- Genere skills desde tus ADRs (pipeline ADR→Skill).
- Active skills automáticamente (router + hooks) según intención/archivo/contenido.
- Haga cumplir CLOOP con Planning Mode duro (sin plan aprobado no se ejecuta).
- Integre Zen-Hub (MCP): filesystem, git, memoria, PM2, métricas, evaluadores.
- Entregue guardrails (DB/seguridad/forma), QA post-respuesta, PM2 para backend y KPIs hacia BMCC/S-Framework.
- Incluya una biblioteca de skills que cubra lo que suelen ofrecer herramientas de pago (p. ej., mapeo de repo, auditorías, scaffolding de tests, PRs, hot-spots, etc.) —en local.

---

2) Topología mínima en el monorepo

```
/.claude/skills/            # (runtime del editor) skills activos
/docs/skills/               # fuentes: SKILL.md + resources + scripts
/docs/adr/                  # ADRs (origen de patrones)
/tools/hooks/               # hooks: pre-invoke (router) y stop (QA)
/tools/adr-pipeline/        # agentes ADR→Skill (miner/curator/builder/evaluator/writer)
/dev/active/                # tríada por tarea: plan.md, context.md, tasks.md
/scripts/pm2/               # ecosystem.config.cjs + playbooks ops
/ci/skills/                 # validadores de bundles y adherencia
/obs/kpi/                   # eventos KPI (jsonl) + export opcional
/bin/                       # CLI (slash-commands: /plan, /plan-save, /devdocs-update)
```

> Nota de nombres: usamos alias neutros (nada de marcas). “editor/IDE” = tu asistente en Cursor.

---

3) Contratos base (listos para pegar)

3.1

skill-rules.schema.json

(router de activación por intención/archivo/contenido)

```
{
```

```
  "$schema":"http://json-schema.org/draft-07/schema#",
  "title":"SkillRules",
  "type":"object",
  "patternProperties":{
    "^[a-z0-9-]+$":{
      "type":"object",
      "required":["type","enforcement","priority"],
      "properties":{
        "type":{"enum":["guideline","guardrail","workflow","analyst","generator"]},
        "enforcement":{"enum":["suggest","require","block"]},
        "priority":{"enum":["critical","high","normal","low"]},
        "promptTriggers":{"type":"object","properties":{
          "keywords":{"type":"array","items":{"type":"string"}},
          "intentPatterns":{"type":"array","items":{"type":"string"}}
        }},
        "fileTriggers":{"type":"object","properties":{
          "pathPatterns":{"type":"array","items":{"type":"string"}},
          "contentPatterns":{"type":"array","items":{"type":"string"}}
        }},
        "resources":{"type":"array","items":{"type":"string"}}
      }
    }
  },
  "additionalProperties":false
}
```

3.2

SKILL.template.md

(≤400 líneas, progresivo)

```
---
name: <skill-id>
type: guideline|guardrail|workflow|analyst|generator
severity: suggest|require|block
version: 0.1.0
---

# Objetivo
Cuándo usarlo y qué resuelve.

# Procedimiento mínimo
1) ...
2) ...
```

Checklist (DoD)
- [ ] Regla A
- [ ] Regla B

Scripts reales
- `pnpm -C <svc> test`
- `node scripts/test-auth-route.js <url>`

Ejemplos
```ts
// OK ...
// NO ...
```

Recursos

- ./resources/01-…md
- ./resources/02-…md

---

4) Hooks del editor (router + QA “no-mess-left-behind”)

4.1 `tools/hooks/userPromptSubmitHook.ts` (pre-invoke)
- Lee `skill-rules.json`.
- Detecta **intención** (keywords/regex), **ruta** (path activo en Cursor) y **contenido** (patrones simples).
- **Inyecta** una nota compacta al sistema, p. ej.:

🎯 SKILL ACTIVATION CHECK:

- backend-dev-guidelines (reason: path: services/api/**)
- database-verification (reason: content: “prisma.”)

→ Cargar main del SKILL y recursos on-demand.

4.2 `tools/hooks/stopHook.ts` (post-respuesta)
Pipeline:
1) **Edit log** → repos tocados.
2) **Prettier** → formateo de archivos editados.
3) **TypeCheck/Build** por repo tocado (`tsc --noEmit`).
4) **Hints**: listar errores TS (si 1–4), recordatorio de manejo de errores (logger/Sentry, BaseController, try/catch).
5) **Auto-resolver**: si ≥N errores → sugerir agente auto-fix; **bloquear merge** si no pasa.

6) **Emite eventos KPI** (`/obs/kpi/events.jsonl`).

---

5) Planning Mode duro (CLOOP obligatorio)

5.1 Slash-commands (CLI)
- `/plan "<tarea>"` → invoca skill **plan-architect** (generator) con tu meta-prompt CLOOP.
- `/plan-save` → skill **plan-save-workflow** guarda tríada
`/dev/active/<tarea>/{plan,context,tasks}.md` + **snapshot MemTech (L1)**.
- `/devdocs-update` → actualiza tríada antes de compaction.

5.2 Regla de bloqueo
El pre-invoke **rechaza** edición/ejecución si **no** existe `plan.md` **aprobado** para la tarea. Devuelve CTA con `/plan` → `/plan-save`.

---

6) Integración MCP (Zen-Hub) y MemTech

MCP/Tools recomendados (nombres neutrales):
- `mcp__fs__*` (crear `/\.claude/skills/...`, escribir `SKILL.md`, `scripts/`).
- `mcp__git__*` (branch `skills/<id>`, commit, PR).
- `mcp__memtech__*` (snapshots L1-L3, admission, tags, link a ADR).
- `mcp__pm2__*` (logs/restart/monit).
- `mcp__eval__*` (tests/benchmarks de adherencia).
- `mcp__metrics__*` (emitir KPIs a BMCC/S-Framework).

Skills MemTech (lista inicial):
- `memory.snapshot` (L1/L2/L3 con hash)
- `memory.admit` (admission policy; rechaza si no cumple esquema)
- `memory.write` (nota persistente; tipo, tags, TTL)
- `memory.query` (búsqueda jerárquica, filtros)
- `memory.link-adr` (vincula plan/skill a ADR)
- `memory.evidence-bundle` (paquetes con citas/metadatos para BMCC)

---

7) Pipeline ADR→Skill (agentes orquestados)

1) **ADR-Miner**
   - Extrae de `/docs/adr/*.md`: contexto, decisión, consecuencias, reglas, DoD, anti-patrones.
   - Emite `skill-candidates.json` (dominio, triggers, checklists, scripts, severidad).

2) **ADR-Curator**
   - Clasifica en `guideline | guardrail | workflow | analyst | generator`.
   - Ensambla `SKILL.md` conciso + `resources/*.md` + **scripts reales** (no simulados).

3) **Skill-Builder (MCP)**
   - Materializa `/.claude/skills/<id>/...`.
   - Actualiza `skill-rules.json` (valida con schema).
   - Abre PR `skills/<id>`.

4) **Skill-Evaluator**
   - Corre 3 escenarios reales; mide activación/adherencia/latencia/errores; emite KPIs.

5) **ADR-Writer**
   - Genera ADR “Adopción Skill <id> vX.Y” con evidencia KPI y DoD.

---

8) Biblioteca de Skills (mínimo viable + “equivalentes” a herramientas de pago)

8.1 Generadores / Workflows (CLOOP)
- `plan-architect` (generator) — plan CLOOP detallado.
- `plan-save-workflow` (workflow) — tríada + snapshot MemTech.
- `testing-plan-designer` (generator) — matriz de pruebas mínimas + comandos.
- `pm2-monitor` (workflow) — `logs/restart/monit` sobre servicios.

8.2 Guidelines (estilo de código y arquitectura)
- `frontend-dev-guidelines` — React/TanStack/UI/estado/accesibilidad.
- `backend-dev-guidelines` — controladores→servicios→repos→DTOs→errores.
- `api-contracts-guidelines` — OpenAPI, versionado, backward-compat.

8.3 Guardrails (bloqueantes o requeridos)
- `database-verification` (**block**) — evita queries inseguras: `findMany`/`update` sin `where/select`, `deleteMany` sin confirmación.
- `secrets-and-config` (**require**) — impide credenciales en código, valida `.env.example`.
- `migration-safety` (**require**) — frena migraciones destructivas sin plan de rollback.

8.4 Analysts (auditorías, “equivalentes” locales a herramientas de pago)
- `repo-auditor` — mapa del repo, deuda técnica, puntos calientes (complejidad/duplicación/archivos grandes), recomendaciones priorizadas.
- `pr-author` — resumen de PR, impacto, breaking changes, checklist.
- `test-scaffolder` — crea esqueletos de tests unitarios/e2e con convenciones del repo.
- `error-fixer` — bucle detectar-arreglar para errores TS/linters frecuentes.
- `dead-code-pruner` — detecta módulos no referenciados/obsoletos y propone PR.
- `dep-risk-advisor` — auditoría de dependencias (licencias, CVEs, versiones).
- `complexity-heatmap` — genera un breve heatmap (por carpetas) para priorizar refactor.
- `todo-debt-collector` — indexa TODO/FIXME anotados y abre issues/acciones.

> Todos estos skills **usan scripts reales** del repo (p. ej., `eslint`, `tsc`, `depcheck`, `npm audit`, `git log --shortstat`) y **no simulan**.

---

9) Guardrails y PM2

DB guardrails (en `docs/skills/db/guardrails.md` + skill `database-verification`):
- Bloquear `findMany`/`update` sin `where/select`.
- Alertar en `deleteMany` y exigir confirmación (workflow).
- Señalizar migraciones destructivas → exigir `/plan` + plan de rollback.

PM2 (`/scripts/pm2/ecosystem.config.cjs` + skill `pm2-monitor`):
- cluster/fork según servicio, `max_memory_restart`, backoff exponencial, rotación de logs.
- Playbooks: `pm2 logs <svc> --lines 200`, `pm2 restart <svc>`, `pm2 monit`.

---

10) KPIs (BMCC / S-Framework)

Evento por iteración → `/obs/kpi/events.jsonl`:
```json
{
  "ts":"2025-10-29T12:00:01Z",
  "task":"feature-x",
  "skills":["backend-dev-guidelines","database-verification"],
  "activated_by":["path: services/api/**","content: prisma."],
  "adherence":true,
  "errors_ts":1,
  "auto_resolver_used":false,
  "latency_ms":8100,
  "tokens_total":24500,
  "zero_errors_left_behind":true
}
```

Indicadores clave:

- skill_activation_precision/recall
- skill_adherence_rate
- mean_fix_latency_s (≈ MTTR de correcciones)
- zero_errors_left_behind_ratio
- tokens_per_operation
- drift_incidents (MemTech) y Δγ correlacionado (S-Framework)

---

11) CI/Gates

- ci/skills/validate-rules — valida skill-rules.json (schema), rutas de resources y existencia de scripts.
- ci/skills/adherence — corre repo-auditor + code-architecture-reviewer sobre el diff; falla bajo umbral.
- ci/skills/no-mess — tras build, cero errores TS o falla.
- ci/kpi/smoke — calcula KPIs básicos y publica artefacto JSON/Prom.

---

12) Fases de implementación (rápidas y con GO/NO-GO)

F0 — Bootstrap (1 día)

- Crear carpetas, skill-rules.schema.json, SKILL.template.md.
- Hooks mínimos (pre-invoke y stop) cableados al editor.

  GO: el pre-invoke muestra “Skill Activation Check” y el stop formatea/compila.

F1 — Skills base (2 días)

- plan-architect, plan-save-workflow, frontend-dev-guidelines, backend-dev-guidelines, database-verification, pm2-monitor.
- skill-rules.json inicial (keywords + pathPatterns + contentPatterns).

  GO: activaciones correctas en casos de prueba.

F2 — Planning Mode duro (1 día)

- /plan, /plan-save, /devdocs-update + bloqueo sin plan aprobado + snapshot MemTech.

  GO: no se permite edición/ejecución si falta plan aprobado.

F3 — QA StopHook completo (1 día)

- prettier → tsc → hints → auto-resolver opcional → KPIs.

  GO: PRs de prueba sin errores residuales.

F4 — Guardrails + PM2 (1 día)

- DB guardrails y playbooks PM2 operativos.

  GO: queries inseguras bloqueadas en tests; pm2 logs accesible desde skill.

F5 — ADR→Skill (2 días)

- ADR-Miner/Curator/Builder/Evaluator/Writer funcionando.

  GO: 2 skills generados desde ADR con ADR de adopción y KPIs.

F6 — KPIs + CI (1 día)

- Eventos KPI + jobs de adherencia y no-mess.

  GO: dashboard básico y gates activos.

---

13) Happy paths diarios (Cursor/IDE)

HP-A: Nueva feature

1. /plan "Feature X" → revisas → /plan-save.
2. Empiezas por secciones; router activa guidelines.
3. StopHook: prettier → tsc → hints/guardrails.
4. testing-plan-designer + ejecutar tests.
5. code-architecture-reviewer PASS → PR + ADR.

HP-B: Bug backend

1. /plan "Fix mailer timeout" → /plan-save.
2. pm2-monitor consulta logs y sugiere fix.
3. Editas; StopHook verifica; guardrail DB asegura seguridad.
4. PR + ADR.

HP-C: Auditoría Sprint

1. /sprint-prep (chequea backlog/DoD).
2. repo-auditor + complexity-heatmap + dep-risk-advisor.
3. Informe + acciones + ADR de lecciones.

---

14) Backlog inmediato (72 h)

1. Bootstrap + Hooks + Planning Mode (F0/F2).
2. 5–7 skills base + skill-rules.json inicial (F1).
3. StopHook completo + guardrail DB + PM2 (F3/F4).
4. ADR→Skill v1 (F5) y KPIs mínimos (F6).

¿Quieres que te entregue el

starter pack

(archivos semilla) en el siguiente mensaje —con

skill-rules.json

inicial, 3 skills base (plan, backend, DB), hooks vacíos,

ecosystem.config.cjs

y 3 slash-commands— listo para copiar al repo?

¡Anotado! Desde ahora no usaré nombres de empresas en archivos, prompts ni roles. Vamos con alias neutros y consistentes.

Propuesta rápida de convención

- Roles base: Orquestador, Planificador, Auditor, Revisor de Código, Ejecutante, Investigador, Curador de ADR, Editor de Docs, Monitor.
- Archivos:
    - docs/roles/orquestador.md
    - docs/roles/auditor.md
    - docs/roles/planificador.md
    - docs/skills/frontend-guidelines.skill.md
    - docs/skills/backend-guidelines.skill.md
- Repos/carpetas: agents/orquestador/, agents/auditor/, agents/ejecutante/, skills/, hooks/, guardrails/.

Mapeo sugerido (si hoy existiera)

- CLAUDE.md → docs/roles/orquestador.md
- BEST_PRACTICES.md → docs/skills/backend-guidelines.skill.md (y dividir por dominios)
- “Claude Code Hooks” → hooks/ide-hooks/ o hooks/editor/
- “Skills de X” → docs/skills/<dominio>.skill.md (+ recursos en docs/skills/<dominio>/resources/)

Snippet opcional para refactor seguro (texto a neutro)

(ejecutar en copia/branch)

```
# 1) Renombrar archivos comunes
git mv CLAUDE.md docs/roles/orquestador.md 2>/dev/null || true

# 2) Reemplazos en contenido (macOS BSD sed):
grep -Irl 'Claude' . | xargs sed -i '' 's/Claude/Agente/g'

# Linux (GNU sed):
# grep -Irl 'Claude' . | xargs sed -i 's/Claude/Agente/g'
```

Si quieres, preparo un pequeño styleguide.md con reglas de nomenclatura (snake/kebab-case, sufijos .skill.md, plantillas de ADR/skill/hook) y una checklist para evitar que el editor “salte” a rutas reales.

¡Excelente base! Con tu “Estado del Arte (Oct-2025)” ya consolidado, aquí está el paquete ejecutable para integrar Skills + Hooks + Planning Mode + Guardrails + PM2 + KPIs en tu ecosistema (CLOOP/ADR/MemTech/Zen-Hub), con happy paths en Cursor/Claude Code y sin “saltos” ni simulaciones.

---

0) Estructura mínima del entorno (añadir al monorepo)

```
/.claude/skills/              # Skills activos para Claude Code (runtime)
/docs/skills/                 # Fuentes (SKILL.md + resources + scripts)
/docs/adr/                    # ADRs existentes (origen de patrones)
/tools/hooks/                 # Hooks: pre-invoke (router) y stop (QA)
/tools/adr-pipeline/          # Agentes ADR→Skill (miner/curator/builder/evaluator/writer)
/dev/active/                  # Tríada por tarea: plan.md, context.md, tasks.md
/scripts/pm2/                 # ecosystem.config.cjs + playbooks ops
/ci/skills/                   # checks de adherencia + validadores de bundles
/obs/kpi/                     # métrica local + export (jsonl/prom)
/bin/                         # CLI (slash-commands, wrappers)
```

---

1) Contracts que gobiernan todo

1.1

skill-rules.schema.json

(router de activación)

```
{
  "$schema":"http://json-schema.org/draft-07/schema#",
  "title":"SkillRules",
  "type":"object",
  "patternProperties":{
    "^[a-z0-9-]+$":{
      "type":"object",
      "required":["type","enforcement","priority"],
      "properties":{
        "type":{"enum":["guideline","guardrail","workflow","analyst","generator"]},
        "enforcement":{"enum":["suggest","require","block"]},
        "priority":{"enum":["critical","high","normal","low"]},
        "promptTriggers":{
          "type":"object",
          "properties":{
            "keywords":{"type":"array","items":{"type":"string"}},
            "intentPatterns":{"type":"array","items":{"type":"string"}}
          }
        },
        "fileTriggers":{
          "type":"object",
          "properties":{
            "pathPatterns":{"type":"array","items":{"type":"string"}},
            "contentPatterns":{"type":"array","items":{"type":"string"}}
          }
        },
        "resources":{"type":"array","items":{"type":"string"}}
      }
    }
  },
  "additionalProperties":false
}
```

1.2

SKILL.template.md

(≤400 líneas + recursos)

---
name: <skill-id>
type: guideline|guardrail|workflow|analyst|generator
severity: suggest|require|block
version: 0.1.0

Objetivo
Qué resuelve y cuándo activarlo (señales).

Procedimiento (pasos mínimos)
1) ...
2) ...

Checklist (DoD)
- [ ] Regla A
- [ ] Regla B

Scripts (usables por el agente)
- `pnpm -C <svc> test`
- `node scripts/test-auth-route.js <url>`

Ejemplos mínimos (bien/mal)
```ts
// OK ...
// NO ...
```

Recursos

- ./resources/01-…md
- ./resources/02-…md

---

2) Hooks (TypeScript) — router + QA “no-mess-left-behind”

2.1 `tools/hooks/userPromptSubmitHook.ts` (pre-invoke)
- Lee `skill-rules.json`.
- Detecta **intención** (keywords/regex), **ruta** (path glob activo en Cursor) y **contenido** (patrones simples en buffer abierto).
- Inyecta al sistema una nota compacta:

🎯 SKILL ACTIVATION CHECK:

- backend-dev-guidelines (reason: path: services/api/**)
- database-verification (reason: content: “prisma.”)

  → Cargar SKILL.md (main) y recursos on-demand.

*(Consejo: evita meter recursos pesados aquí; solo el **main** y decide en la respuesta si trae recursos.)*

2.2 `tools/hooks/stopHook.ts` (post-respuesta, QA)
Pipeline:
1) **Edit log** → detectar repos tocados.
2) **Prettier** → formatear solo archivos editados.
3) **Build/TypeCheck** por repo tocado (`tsc --noEmit`).
4) **Hints**: listar errores TS (si 1–4), recordar **manejo de errores** en controladores/repos.
5) **Auto-resolver**: si ≥5 errores → invocar tu agente auto-fix; **bloquear merge** si no pasa.
6) **Emitir eventos KPI** (obs/kpi, ver §5).

Sugerencia de severidades:
- `database-verification` → **block** (corta paso si query insegura).
- `guidelines`/`analyst` → **require** (debe quedar en PASS).
- `workflow` → **require** (plan/plan-save).

---

3) Planning Mode duro (CLOOP “Clarify→Layout→Operate→Observe→Reflect”)

3.1 Slash-commands (CLI o snippets en Cursor)
- `/plan "<tarea>"` → invoca skill **plan-architect** (generator) con tu meta-prompt; salida en Markdown estructurado.
- `/plan-save` → skill **plan-save-workflow** crea `/dev/active/<tarea>/{plan,context,tasks}.md` + snapshot MemTech L1.
- `/devdocs-update` → actualiza tríada antes de compaction.

3.2 Regla de bloqueo
Hook pre-invoke **rechaza** edición/ejecución si **no** existe `/dev/active/<tarea>/plan.md` con `status: Approved`. Emite CTA: “Ejecuta /plan → /plan-save”.

---

4) Guardrails operativos

4.1 DB/Prisma (ejemplo de recurso `docs/skills/db/guardrails.md`)
- **Bloquear** `findMany(`/`update(` sin `where` o `select`.
- **Sugerir** `select` estricto (column minimization).

- **Detectar** `deleteMany` → exigir `where` + confirmación.
- **Marcar** migraciones destructivas → detener y pedir `/plan` de rollback.

4.2 Seguridad/Salida (opcionales)
- Moderación/regex básicas (longitud, JSON válido, secreto accidental).
- Política externa (OPA) si deseas endurecer (puente a futuro).

---

5) Observabilidad y KPIs (BMCC/S-Framework ready)

5.1 Evento único por iteración (JSONL)
Archivo: `/obs/kpi/events.jsonl`
```json
{
  "ts":"2025-10-29T09:00:01Z",
  "task":"feature-x",
  "skills":["backend-dev-guidelines","database-verification"],
  "activated_by":["path:services/api/**","content:prisma."],
  "adherence":true,
  "errors_ts":2,
  "auto_resolver_used":false,
  "latency_ms":8420,
  "tokens_total":27150,
  "zero_errors_left_behind":true
}
```

5.2 KPIs computados (export prom opcional)

- skill_activation_recall/precision
- skill_adherence_rate
- mean_fix_latency_s
- zero_errors_left_behind_ratio
- tokens_per_operation

  (Si usas PM2+tx2, expón gauges/meters y crea alertas).

---

6) PM2 — orquestación backend y ops

6.1

scripts/pm2/ecosystem.config.cjs

(esqueleto)

```
module.exports = {
  apps: [
    { name: "api-users", cwd: "services/api-users", script: "pnpm", args: "start",
      exec_mode: "cluster", instances: 2, max_memory_restart: "512M",
      env: { NODE_ENV: "development" }, out_file: "./logs/out.log", error_file: "./logs/error.log"
},
    { name: "api-mailer", cwd: "services/api-mailer", script: "pnpm", args: "start",
      exec_mode: "fork", max_restarts: 10, exp_backoff_restart_delay: 200,
      env: { NODE_ENV: "development" } }
  ]
}
```

6.2 Skill

pm2-monitor

(workflow)

- Playbooks: pm2 logs <svc> --lines 200, pm2 restart <svc>, pm2 monit.
- Hook post-respuesta permite a Claude leer logs específicos y proponer fix.

---

7) Pipeline ADR→Skill (tus agentes, con tools)

1. ADR-Miner
    - Extrae de /docs/adr/*.md: contexto, decisión, consecuencias, reglas, DoD, anti-patrones.
    - Emite skill-candidates.json (dominio, triggers, checklists, scripts).

2. ADR-Curator
    - Clasifica en guideline|guardrail|workflow|analyst|generator.
    - Ensambla SKILL.md conciso + resources/*.md + scripts reales.
3. Skill-Builder (MCP tools)
    - Materializa /.claude/skills/<id>/....
    - Actualiza skill-rules.json (validado contra schema).
    - Crea rama skills/<id> y PR.
4. Skill-Evaluator
    - Ejecuta 3 escenarios reales; mide adherencia/latencia/errores; registra eventos KPI.
5. ADR-Writer
    - Crea ADR “Adopción Skill vX.Y” con evidencia KPI y DoD.

(Puedo darte el script “adr2skills.mjs” básico si lo quieres listo.)

---

8) Happy Paths (lo que harás en Cursor día a día)

HP-A: Nueva feature

1. /plan "Feature X" → plan detallado.
2. Revisas → /plan-save (tríada + snapshot MemTech).
3. Empiezas a editar por secciones. Pre-invoke activa guidelines.
4. StopHook: prettier → tsc → hints/guardrail DB.
5. /test-plan genera pruebas mínimas; ejecutas.
6. code-architecture-reviewer PASS → PR + ADR.

HP-B: Bug backend

1. /plan "Fix mailer timeout" → /plan-save.
2. pm2-monitor consulta logs; propone fix.
3. Editas; StopHook verifica; guardrail DB mantiene seguridad.
4. PR + ADR “Fix mailer timeout”.

HP-C: Pre-Sprint / Auditoría

1. /sprint-prep valida backlog, riesgos, DoD.
2. checkpoint-audit audita adherencia, cobertura, migraciones.
3. Reporte + acciones + ADR de lecciones.

---

9) CI/CD y gates

- ci/skills/validate-rules: validar skill-rules.json (schema), rutas de recursos y existencia de scripts.
- ci/skills/adherence: correr code-architecture-reviewer sobre el diff; fallar si < umbral.
- ci/skills/no-mess: tras build, cero errores TS o falla.
- ci/kpi/smoke: calcula KPIs básicos del último job y publica artefacto JSON/Prom.

---

10) Kits iniciales de Skills (recomendados)

1. plan-architect (generator) — produce plan CLOOP con tu meta-prompt.
2. plan-save-workflow (workflow) — guarda tríada + snapshot MemTech.

3. frontend-dev-guidelines / backend-dev-guidelines (guideline).
4. database-verification (guardrail; block).
5. code-architecture-reviewer (analyst).
6. testing-plan-designer (generator).
7. pm2-monitor (workflow).

(Todos con recursos breves y scripts reales; nada de “simular”.)

---

11) Antipatrón y mitigación (rápido)

- Skills gigantes → recortar main ≤400 líneas + mover detalles a resources/.
- Autoactivación ruidosa → refinar intentPatterns + pathPatterns (prioridades).
- “Haríamos / Simularíamos” → StopHook detecta lenguaje simulado y exige script/artefacto real.
- Rot de skills → job mensual “skills-audit”: re-evalúa activación/adherencia y abre issues.

¿Qué te dejo listo ahora?

- skill-rules.schema.json (arriba).
- SKILL.template.md (arriba).
- Esqueleto de hooks (qué hacer y en qué orden).
- Ecosystem PM2 base.
- Formato de eventos KPI + lista de KPIs.
- Blueprint del pipeline ADR→Skill (con agentes).

Si quieres, en el próximo mensaje te dejo los archivos semilla (carpetas + skill-rules.json inicial + 3 skills base + hooks vacíos + PM2 config + 3 slash-commands) en formato pegable para tu repo.

¡Vamos! Aquí tienes el plan ejecutable para montar, dentro de tu ecosistema, todo lo necesario para crear y usar Skills de forma rutinaria —con happy paths en Cursor/Claude Code, sin “saltos” ni simulaciones de pasos.

Objetivo

Crear un entorno operativo en tu monorepo para:

- Definir Skills (bundles cortos + recursos + scripts).
- Auto-activar Skills (pre-invoke) según intención, ruta o contenido.
- Hacer cumplir el flujo CLOOP con Plan Mode duro (gates) y StopHook (QA post-respuesta).
- Persistir planes y contexto (dev-docs + MemTech).
- Operar servicios backend con PM2 y guardrails (DB/seguridad).
- Medir adherencia y calidad (BMCC/S-Framework KPIs).

Arquitectura de carpetas (añadir al monorepo)

```
/.claude/skills/            # skills activos para Claude Code
/tools/agent-hooks/         # pre-invoke + stop hooks (router/QA)
/docs/skills/               # bundles fuente (core + resources + scripts)
/docs/adr/                  # ADRs (origen de patrones)
/dev/active/                # plan/context/tasks por tarea (se autogenera)
/scripts/pm2/               # ecosystem.config.cjs
/bin/                       # utilidades CLI (p.ej., cloop-dev.mjs)
```

Fase 0 — Bootstrap (1 día)

Entregables

- tools/agent-hooks/ (router + stop pipeline).
- docs/skills/SKILL.template.md y skill-rules.json semilla.
- bin/cloop-dev.mjs (crear/actualizar tríada dev-docs).
- scripts/pm2/ecosystem.config.cjs.

Tareas

1. Crear carpetas y archivos mínimos.
2. Agregar pnpm scripts:
    - "pm2:start": "pm2 start scripts/pm2/ecosystem.config.cjs".
    - "devdocs:start": "node bin/cloop-dev.mjs start", "devdocs:update": "node bin/cloop-dev.mjs update".
3. Activar hooks desde tu runner/MCP:
    - pre-invoke → llama userPromptSubmitHook e inyecta la nota "🎯 Skill Activation Check".
    - post-invoke → llama stopHook para prettier → tsc(repos tocados) → hints/guardrails → auto-resolver si ≥N errores.

Gate GO

- Hook pre-invoke inyecta nota cuando hay match de triggers.
- StopHook detecta repos tocados y corre tsc + formatea.

Fase 1 — Skill Bundles base (2 días)

Bundles iniciales (mínimo)

- frontend-dev-guidelines (core ≤400 líneas + resources/).

- backend-dev-guidelines.
- database-verification (guardrail bloqueante).
- plan-architect (generator, produce plan detallado con tu metaprompt).
- plan-save-workflow (workflow, guarda tríada plan/context/tasks bajo /dev/active/...).
- architecture-designer (generator).
- testing-plan-designer (generator).
- code-architecture-reviewer (analyst).
- pm2-monitor (workflow para logs/restart/health).

Tareas

1. Convertir 2–3 ADRs clave en resources (reglas + checklist + anti-patrones).
2. Escribir SKILL.md de cada bundle (≤400 líneas), referenciar scripts reales (no generados).
3. Completar skill-rules.json con keywords, intentPatterns, pathPatterns, contentPatterns.

Gate GO

- Cada bundle compila (lint/links), y el router lo activa en prompts y/o archivos esperados.

Fase 2 — Plan Mode duro + Tríada dev-docs (1 día)

Objetivo

Nadie edita/ejecuta hasta que el plan esté creado y aprobado.

Tareas

1. Slash-commands:
    - /plan → invoca plan-architect.
    - /plan-save → invoca plan-save-workflow + devdocs:start <tarea>.
2. Hook pre-invoque y/o tu CLI:
    - Si no existe dev/active/<tarea>/plan.md aprobado, rechazar acciones de edición/ejecución (mensajes de corrección).
3. Integrar con MemTech:
    - Al salvar plan, crear snapshot L1 (índice + hash) para anti-drift.

Gate GO

- Intento de editar sin plan aprobado → bloqueo con explicación y CTA /plan.

Fase 3 — QA automática (StopHook completo) (1 día)

Pipeline

prettier → tsc por repos tocados → error-hints → (si errores ≥5) recomendar auto-resolver

Tareas

- Parsea edit-log (archivos y repos).
- Aplica prettier a editados.
- tsc --noEmit por repo tocado; cuenta errores TS.
- Muestra hints de errores y recordatorio de manejo de errores (Sentry/log + BaseController).
- Si ≥ N errores → proponer agente auto-resolver (tu MCP) y bloquear merge.

Gate GO

- PRs de prueba sin errores residuales tras StopHook (objetivo “zero-errors-left-behind”).

Fase 4 — Guardrails críticos + PM2 (1 día)

Tareas

- database-verification: bloquear findMany(/ update(sin where/select.
- pm2-monitor: exponer comandos pm2 logs <svc>, pm2 restart <svc>, pm2 monit.
- Playbooks de troubleshooting (cortos) en resources/ops/.

Gate GO

- 0 queries inseguras en 1 sprint de prueba.
- MTTD backend < 2 min gracias a pm2 logs.

Fase 5 — Agentes ADR integrados al pipeline Skills (2 días)

Flujo

1. ADR-Miner → extrae patrones de docs/adr/*.md → skill-candidates.json.
2. ADR-Curator → materializa SKILL.md + resources/ + scripts/.
3. Skill-Builder (con tools MCP) → crea carpetas en /.claude/skills/ y actualiza skill-rules.json.
4. Skill-Evaluator → corre casos reales (3 escenarios), recoge adherencia/latencia/errores.

5. ADR-Writer → genera ADR “Adopción Skill X” + versionado.

Gate GO

- 2 Skills generados desde ADR y adoptados con ADR de respaldo.

Fase 6 — KPIs y CI/Quality Gates (1 día)

KPIs BMCC/S-Framework

- skill_activation_recall/precision
- skill_adherence_rate
- zero_errors_left_behind
- mean_fix_latency (s)
- tokens_per_operation / latency_ms
- drift_incidents (MemTech) y Δγ correlacionado

CI

- Job “skills-check”: valida skill-rules.json, enlaces de resources/, scripts existentes.
- Gate de merge: adherencia ≥ umbral en PR (revisión con code-architecture-reviewer).

Gate GO

- Dashboard con 4 KPIs visibles; falla la pipeline si adherencia < umbral.

Happy Paths (pasos exactos)

HP-1: Nueva Feature (frontend+backend)

1. En Cursor: /plan "Feature X" → agente genera plan (sin editar aún).
2. Apruebas plan → /plan-save → crea /dev/active/feature-x/{plan,context,tasks}.md + snapshot MemTech.
3. Editas por secciones; router activa frontend/ backend guidelines.
4. StopHook formatea/compila; si errores, muestra hints / dispara auto-resolver.
5. testing-plan-designer genera pruebas mínimas; ejecútalas.
6. code-architecture-reviewer pasa; PR listo.
7. ADR de adopción/decisiones actualizada.

HP-2: Bugfix servicio backend

1. /plan "Fix email timeouts" → /plan-save.
2. pm2-monitor → pm2 logs email --lines 200 (agente lo ejecuta).
3. Aplicas fix en repo; StopHook asegura cero errores.
4. database-verification garantiza queries seguras.
5. PR + ADR “Fix email timeouts”.

HP-3: Pre-Sprint / Auditoría

1. /sprint-prep → revisa backlog, riesgos, DoD.
2. checkpoint-audit audita skills activos, cobertura, migraciones.
3. Reporte + acciones correctivas y ADR.

Mecanismos anti-“simular pasos”

- Bloqueo Plan Mode: sin plan.md aprobado en /dev/active/<tarea>/ → no se permite edición/ejecución.

- StopHook: si detecta lenguaje de simulación (“haría”, “deberíamos”) + falta de artefactos (archivos/commits), emite FAIL y CTA a scripts reales.
- Guardrails: bloqueos duros en DB/seguridad.
- Commit-gates: no merge si falta la tríada dev-docs actualizada en la tarea.

DoD / GO-NO-GO por fase

- F0: Hooks cableados y skill-rules.json cargando → GO.
- F1: 5 skills base auto-activan con triggers → GO.
- F2: Plan Mode bloquea edición sin plan → GO.
- F3: “Zero-errors-left-behind” en PRs de prueba → GO.
- F4: Guardrail DB bloquea unsafe queries (tests) → GO.
- F5: 2 skills generados desde ADR + ADR de adopción → GO.
- F6: Dashboard KPIs y gate de adherencia en CI → GO.

Riesgos y mitigación

- Falsos positivos de activación → ajustar intentPatterns/pathPatterns; usar “priority” y “enforcement”.
- Rot de skills → revisión mensual automatizada (agente Evaluator + reporte).
- Coste de tokens → core ≤400 líneas + resources; cachear/evitar recursos innecesarios.
- Lock-in → contratos neutrales (JSON/TS) + scripts de sistema (PM2, tests) reutilizables con cualquier LLM/IDE.

Backlog inmediato (72 h)

1. F0/F2: Hooks + Plan Mode + tríada dev-docs (slash-commands).
2. F1: 5 skills (front/back/DB/plan/pm2) + skill-rules.json.
3. F3/F4: StopHook completo + guardrail DB.
4. F6: KPIs mínimos y gate de adherencia en CI.

¿Quieres que te deje el starter pack (carpetas + archivos base + seeds de skills + hooks vacíos + scripts PM2 + slash-commands) listo para pegar en tu repo?

¡Sí, y buen ojo! Podemos “enchufar” tus agentes ADR al flujo de Skills sin perder nada de tu disciplina CLOOP. Primero, un resumen de la guía oficial de Anthropic (para alinear lenguaje y límites), y luego el plano de integración con tus agentes + tools.

Qué dice (en corto) la guía oficial de Skills

- Qué son: carpetas con SKILL.md + (opcional) recursos y scripts. Claude las carga cuando son relevantes, de forma progresiva (metadatos → instrucciones → recursos/código) para ahorrar contexto. Funcionan en Claude apps, Claude Code y API (con matices).
- Auto-invocación y composición: Claude escanea Skills disponibles, activa las pertinentes y puede apilar varias en una misma tarea. Incluso “verás” el uso en su razonamiento (“chain of thought”).
- Dónde:
    - Claude apps (Pro/Max/Team/Enterprise), incluye Skills preconstruidas (PPTX/XLSX/DOCX/PDF).
    - Claude Code: Skills custom en el filesystem (~/.claude/skills o en el repo).
    - API: headers beta + endpoint /v1/skills; Skills no se sincronizan automáticamente entre superficies (hay que subirlas por separado).
- Autoría y buenas prácticas: concisas, bien estructuradas, evaluadas con casos reales; SKILL.md con front-matter (name, description) y divulgación progresiva; añadir scripts utilitarios cuando la operación sea frágil (p.ej., migraciones).

- Hooks en Claude Code: puedes inyectar contexto en UserPromptSubmit, bloquear/permitir acciones, y hasta dirigirte a herramientas MCP por nombre (útil para tu Memory MCP y Zen Hub).

¿Podemos usar los mismos agentes ADR para crear Skills dentro del pipeline ADR?

Sí. De hecho, es el encaje natural: tus ADR ya capturan patrones, decisiones y DoD; los agentes ADR pueden minar esos patrones y emitir Skills concisas + recursos de referencia. El pegamento son hooks + tools MCP.

Diseño propuesto (CLOOP → Skills)

1. Clarify (agente ADR-Miner)
    - Lee ADRs/plantillas, extrae: intenciones, reglas, anti-patrones, DoD, scripts asociados.
    - Output: skill-candidates.json (dominio, triggers, checklists, scripts).
2. Layout (agente ADR-Curator)
    - Mapea candidatos a tipos de Skill: guideline, workflow, guardrail, analyst, generator.
    - Ensambla SKILL.md conciso + recursos adjuntos (p.ej., REFERENCE.md, CHECKS.md, scripts/), siguiendo divulgación progresiva.
3. Operate (agente Skill-Builder con tools)
    - Usa tools MCP (filesystem/git) para materializar /.claude/skills/<skill>/ y actualizar skill-rules.json.
    - Opcional: empaqueta variante API y variante Claude.ai (recordando que no se sincronizan; se suben aparte).
4. Observe (agente Skill-Evaluator)

    - Corre evaluaciones mínimas por modelo (Haiku/Sonnet/Opus) y casos reales (tu backlog), midiendo activación, adherencia, latencia.

5. Reflect (agente ADR-Writer)
    - Si el Skill pasa, genera/actualiza un ADR (“Adopción de Skill X”), y versiona el Skill (tag + CHANGELOG).

Nota: Los hooks de Claude Code te permiten:
a) Inyectar recordatorios/activaciones en UserPromptSubmit;
b) Tras cada respuesta (Stop), formatear, compilar, reportar errores y sugerir agentes auto-fix;
c) Dirigirte a MCP tools por patrón (mcp__memory__…, mcp__filesystem__…).

Orquestación con tus agentes + tools (mínimo viable)

- Tools MCP:
    - mcp__filesystem__* (crear carpetas /\.claude/skills/…, escribir SKILL.md, scripts/).
    - mcp__git__* (branch skills/<feature>, commit, PR).
    - mcp__memory__* (indexar Skills y enlazarlos a decisiones previas).
    - mcp__eval__* (correr tests/benchmarks de adherencia).
- Hooks (Claude Code):
    - UserPromptSubmit: inyectar “Skill Activation Check” si el prompt coincide con triggers del repositorio (palabras clave, paths, patrones de contenido).
    - PostToolUse/Stop: prettier → build/tsc de repos tocados → reporte; si ≥N errores, sugerir tu agente auto-resolver. (Patrón soportado por la guía de hooks).

Qué gana cada lado (diferencias funcionales)

- Skills (Anthropic) te dan auto-invocación, carga progresiva y portabilidad entre productos (apps/Code/API) — con la salvedad de subidas separadas por superficie.

- Tu ecosistema (CLOOP + ADR + MCP) aporta gobernanza, DoD auditable, hooks de calidad y memoria organizacional (ADRs) para generar y versionar Skills; además, puedes amarrar scripts productivos (PM2, tests, migraciones) como recursos del Skill, como recomienda Anthropic para operaciones frágiles.

Pasos tácticos (hoy mismo)

1. Extractor ADR → Skill: agente lee ADRs y genera /.claude/skills/<id>/SKILL.md + resources/ + scripts/ (conciso).

2. Router de activación: JSON skill-rules.json (keywords, regex de intención, pathPatterns, contentPatterns) consumido por hook UserPromptSubmit.

3. Pipeline de publicación:

    - Claude Code: se instala directo en filesystem.

    - API/Claude.ai: empaquetar y subir por separado (mismo contenido, distintos “destinos”).

4. Evaluación y ADR: agente evaluador corre pruebas (3 modelos + escenarios reales) y escribe ADR de adopción si pasa.

Si te late, preparo el esqueleto de hooks + carpetas /.claude/skills/ + plantillas SKILL.md y el agente “ADR→Skill” para que lo pegues en tu monorepo y lo corras con tus MCP.

¡Perfecto! Tomemos tus ADRs como “mina de oro” para generar Skills que sigan tu flujo CLOOP, y sumemos skills “analistas” al estilo Coder Rabbit (checkpoints, pre-sprints, auditorías). Aquí tienes un método completo + artefactos listos para empezar hoy.

---

1) Método “ADR → Skills” (en 5 pasos)

Paso A — Descubrir patrones (minería de ADRs)

- Extrae de cada ADR: Contexto, Decisión, Consecuencias, Reglas/Normas, Checks/DoD, Anti-patrones.
- Normaliza en una tabla: dominio, intención, paths, patrones de contenido, checklist, riesgos.

Paso B — Taxonomía de Skills

- guideline (guía práctica: frontend, backend, testing).
- guardrail (bloqueante o de alta severidad: BD/Prisma, seguridad, compliance).
- workflow (pasos operativos: plan→save→implement→review→retro).
- analyst (revisores automáticos: arquitectura, PRs, auditorías, pre-sprint).
- generator (plantillas: plan detallado, arquitectura, plan de pruebas).

Paso C — Plantilla SKILL.md (≤400 líneas + recursos)

- Qué hace (objetivo, alcance, “cuándo activarlo”).
- Cómo (patrón paso a paso, ejemplos mínimos).
- Checklist (DoD, pruebas, seguridad).
- Scripts adjuntos (comandos concretos que el agente debe usar).
- Referencias internas (enlaza a recursos docs/skills/... breves).

Paso D — Diseño de disparadores (triggers)

- keywords (intención: “plan”, “architecture”, “testing”, “audit”).
- intentPatterns (regex de acciones: (crear|plan|design|test|audit)).
- pathPatterns (globs por módulo: frontend/**, services/**/src/**).
- contentPatterns (marcadores: @prisma/client, describe(, router.).

Paso E — Wiring

- Pre-invoke: SkillRouter inyecta “🎯 Skill Activation Check”.
- Post-respuesta: StopHook corre prettier → tsc (repos tocados) → hints/guardrails → auto-resolver si ≥N errores.
- Plan Mode: bloquear edición hasta PASS de plan; al aprobar, skill workflow guarda el plan en /dev/active/<tarea>/{plan,context,tasks}.md.

---

2) Starter: plantilla de SKILL + generador desde ADR

2.1 SKILL.template.md (núcleo)

```
# Skill: <nombre>
Tipo: guideline|guardrail|workflow|analyst|generator
Severidad: suggest|require|block
Objetivo: <qué problema resuelve>
Cuándo usarlo: <escenarios>

## Procedimiento
1) ...
2) ...

## Checklist (DoD)
- [ ] Requisito A
- [ ] Requisito B

## Scripts
- `pnpm pm2:start`
- `node scripts/test-auth-route.js <url>`

## Ejemplos mínimos
```ts
// breve snippet correcto/incorrecto
```

Recursos

- ./resources/01-…md
- ./resources/02-…md
```

2.2 Semilla `skill-rules.json` con tus nuevos skills

```json
{
  "plan-architect": {
    "type": "generator",
    "enforcement": "require",
    "priority": "high",
    "promptTriggers": { "keywords": ["plan","planning","estrategia","roadmap"] },
    "fileTriggers": { "pathPatterns": ["dev/active/**"] },
    "resources":
["docs/skills/plan/PLAN.template.md","docs/skills/plan/resources/checklist.md"]
  },
  "plan-save-workflow": {
    "type": "workflow",
    "enforcement": "require",
    "priority": "high",
    "promptTriggers": { "intentPatterns": ["(approve|aceptar).*plan","salir.*plan mode"] },
    "resources": ["docs/skills/workflow/save-plan.md"]
  },
  "architecture-designer": {
    "type": "generator",
    "enforcement": "suggest",
    "priority": "high",
    "promptTriggers": { "keywords": ["arquitectura","diagrama","módulos"] },
    "fileTriggers": { "pathPatterns": ["frontend/**","services/**/src/**"] },
    "resources": ["docs/skills/arch/ARCH.template.md"]
  },
  "testing-plan-designer": {
    "type": "generator",
    "enforcement": "suggest",
    "priority": "normal",
    "promptTriggers": { "keywords": ["testing","pruebas","coverage","jest","playwright"] },
    "resources": ["docs/skills/testing/TESTPLAN.template.md"]
  },
  "code-architecture-reviewer": {
    "type": "analyst",
    "enforcement": "require",
    "priority": "high",
    "fileTriggers": { "pathPatterns": ["**/*.{ts,tsx}"] },
    "resources": ["docs/skills/review/architecture-checks.md"]
  },
  "sprint-precheck": {
    "type": "analyst",
    "enforcement": "require",
    "priority": "high",
    "promptTriggers": { "keywords": ["pre-sprint","checkpoint","kickoff"] },
    "resources": ["docs/skills/review/pre-sprint-checklist.md"]
  },
```

```
  "checkpoint-audit": {
    "type": "analyst",
    "enforcement": "require",
    "priority": "high",
    "promptTriggers": { "keywords": ["audit","auditoría","compliance"] },
    "resources": ["docs/skills/review/audit-checks.md"]
  },
  "pm2-monitor": {
    "type": "workflow",
    "enforcement": "suggest",
    "priority": "normal",
    "promptTriggers": { "keywords": ["pm2","logs","restart","monit"] },
    "resources": ["docs/skills/ops/pm2.md"]
  },
  "database-verification": {
    "type": "guardrail",
    "enforcement": "block",
    "priority": "critical",
    "fileTriggers": {
      "pathPatterns": ["**/*.prisma","services/**/src/repositories/**/*.ts"],
      "contentPatterns": ["prisma\\.","findMany\\(","update\\("]
    },
    "resources": ["docs/skills/db/guardrails.md"]
  }
}
```

2.3 Generador “ADR → Skill resources” (mínimo viable)

Guárdalo como /tools/adr2skills.mjs y ejecútalo con node:

```
#!/usr/bin/env node
import fs from "node:fs/promises";
import path from "node:path";

const ADR_DIR = "docs/adr";
const SKILLS_DIR = "docs/skills/_from-adr";

const pick = (txt, h) => {
  const re = new RegExp(`^##\\s*${h}\\s*\\n([\\s\\S]*?)(?=^##\\s|$)`, "m");
  const m = txt.match(re);
  return m ? m[1].trim() : "";
};

const toSlug = s => s.toLowerCase().replace(/[^a-z0-9]+/g,"-").replace(/^-|-$/g,"");

const tpl = (name, ctx, rules, checklist) => `# From ADR: ${name}
Tipo: guideline
```

```
Objetivo: Derivado de ADR para estandarizar práctica.

## Contexto (resumen)
${ctx || "-"}

## Reglas Clave
${rules || "-"}

## Checklist
${checklist || "-"}
`;

async function main(){
  await fs.mkdir(SKILLS_DIR, { recursive:true });
  const files = (await fs.readdir(ADR_DIR)).filter(f=>f.endsWith(".md"));
  for(const f of files){
    const raw = await fs.readFile(path.join(ADR_DIR,f), "utf8");
    const name = (raw.match(/^#\s*(.*)$/m)?.[1] || f).trim();
    const ctx = pick(raw,"Contexto|Context");
    const dec = pick(raw,"Decisión|Decision");
    const cons = pick(raw,"Consecuencias|Consequences");
    const rules = [dec, cons].filter(Boolean).join("\n\n");
    const checklist = pick(raw,"Checklist|DoD|Hechos");
    const out = tpl(name, ctx, rules, checklist);
    const slug = toSlug(name);
    await fs.writeFile(path.join(SKILLS_DIR, `${slug}.md`), out);
  }
  console.log("✅ Recursos de skills generados desde ADR.");
}
main().catch(e=>{ console.error(e); process.exit(1); });
```

Esto te crea resources iniciales a partir de ADRs (no el SKILL.md principal). Luego los enlazas desde el SKILL principal correspondiente.

---

3) Integración con CLOOP y

Plan Mode

(Cursor)

Clarify → plan-architect (generator) produce plan estructurado (usa tus metaprompts)

Layout → architecture-designer + testing-plan-designer crean blueprint y plan de pruebas

Operate → frontend-dev-guidelines, backend-dev-guidelines, database-verification (guardrail)

Observe → StopHook: prettier → tsc → hints → auto-resolver + pm2-monitor

Reflect → checkpoint-audit registra hallazgos, actualiza ADRs y retro con KPIs

Workflow complementario

- Al aprobar el plan en Plan Mode: activa plan-save-workflow → guarda en /dev/active/<tarea>/.

- Slash-commands sugeridos:

  /plan (genera plan), /plan-save, /arch-gen, /test-plan, /build-and-fix, /audit-pr, /sprint-prep.

---

4) Skills “analistas” tipo Coder Rabbit (mímica controlada)

code-architecture-reviewer

- Checks: capas (rutas→controladores→servicios→repos), errores TS, side-effects, contratos de tipos, cohesión.

sprint-precheck

- Verifica: backlog grooming hecho, planes/recursos listos, riesgos listados, DoD definidos, test plan mínimo.

checkpoint-audit (medio sprint / PR)

- Confirma: adherencia a skills activos, cobertura de pruebas, cambios de schema documentados, scripts ejecutados.

Todos con DoD explícito y scripts adjuntos (p.ej., pnpm -C services/foo test, pm2 logs <svc>), para que el agente use herramientas existentes.

---

5) Guardrails & QA (rápidos de alto impacto)

- database-verification (block): detecta findMany(/update(sin where/select → detiene edición o exige corrección.
- error-handling reminder (suggest): si ve controladores/repos, recuerda try/catch + captura (Sentry/log) y base controller.
- pm2-monitor (workflow): troubleshooting estándar (pm2 logs, pm2 restart, pm2 monit) con mini-playbooks.

---

6) KPIs y pruebas A/B (enchufar a tu BMCC)

- skill_activation_recall / precision
- skill_adherence_rate (% respuestas que cumplen el skill activo)
- zero_errors_left_behind (% sin errores tras StopHook)
- mean_fix_latency (s)
- tokens_per_operation / latencia
- incident_drift (Memoria) correlacionado con adherencia

Experimento A/B (2 semanas)

- A: sin Skills nuevos | B: con SkillRouter + StopHook + 4 skills clave.

- PASS si: +25% adherence, +90% zero-errors, -30% mean_fix_latency.

---

7) Roadmap sugerido (7–10 días)

Día 1–2: Minería ADR → resources; crear plan-architect, plan-save-workflow, architecture-designer, testing-plan-designer.

Día 3–4: SkillRouter pre-invoke + StopHook completo; database-verification.

Día 5–6: Analistas (code-architecture-reviewer, sprint-precheck, checkpoint-audit) + PM2 skill.

Día 7–10: A/B con KPIs, ajuste de triggers, consolidar plantillas SKILL.md y slash-commands.

¿Quieres que te deje un pack con los archivos base (SKILL.template.md, skill-rules.json semilla, adr2skills.mjs, 4 skills iniciales y 3 analistas) listo para pegar en tu monorepo?

¡Buen punto! Para evitar “contaminar” búsquedas abiertas con tus conceptos CLOOP / BMCC / S-Framework (que no existen en la web) te dejo un paquete anti-alucinaciones: política de investigación, prompt de deep-research revisado y utilidades para sanear/encapsular términos privados.

---

1) Política SRP (Safe Research Policy) v1.0

- Prohibido en web/open-search: CLOOP, BMCC, S-Framework, MemTech, Zen Hub Agent, ClaimTrace y derivados.

- Doble carril

    - Carril Público (web): solo términos genéricos y dominios oficiales (docs de skills/hooks/IDE planning, PM2, Prisma, OPA, telemetría).

    - Carril Privado (local/RAG): integración con tus conceptos usando tus docs internos (MemTech L1-L3, ADRs, meta-prompts).

- Puente controlado: mapeo término-neutro ↔ término-privado en un glosario local (no se sube a la web).
- Citas obligatorias en todo lo público; nunca comparar o inferir definiciones de tus siglas desde fuentes externas.
- Pruebas anti-alucinación: checklist por sección (¿se citó?, ¿se filtraron siglas?, ¿hay inferencias no sustentadas?).

---

2) Glosario puente (local)

Concepto privado	Alias neutro para búsquedas públicas
CLOOP	“ciclo de planificación-ejecución con gates de aprobación”
BMCC	“calibración metacognitiva/epistémica de agentes con métricas de adherencia”
S-Framework	“esquema sorpresa→modulación→ajuste de representación/embedding”
Zen Hub Agent	“orquestador multi-agente con políticas”
MemTech (MCP memoria)	“memoria jerárquica/RAG interno con anti-drift”
ClaimTrace/OPA	“trazabilidad y políticas declarativas de cumplimiento”

> Este glosario solo se usa en tu carril privado para enlazar hallazgos públicos con tus conceptos.

---

3) Prompt de Deep-Research (revisado, seguro para la web)

Pégalo tal cual en tu agente investigador. Diseñado para no filtrar tus nombres privados a internet.

DeepResearch (SAFE) — Integración de skills/hooks/plan/guardrails/PM2/KPIs en ECO-S
ROLE: >
  Investigador Técnico Senior + Arquitecto de Plataformas de IA (agentic coding / IDE hooks / QA / MLOps).
MISSION: >
  Construir una base teórico-técnica robusta para integrar "skills + hooks + plan mode + guardrails + PM2 + KPIs"
  en un ecosistema privado existente (con ciclo de planificación-gates, memoria jerárquica local, orquestación de agentes y
  gobernanza/telemetría). La síntesis final NO debe mencionar ni inferir nombres o siglas propietarios.

SAFEGUARDS:
  - NO usar ni mencionar: "CLOOP", "BMCC", "S-Framework", "MemTech", "Zen Hub Agent", "ClaimTrace".
  - NO intentar deducir significados de esos términos a partir de fuentes públicas.
  - SEPARAR carriles:
      - Público: documentación oficial y papers/demos sobre "skills", "IDE hooks", "planning mode", "guardrails", "PM2", "DB safety (Prisma)", "OPA", "telemetría/KPIs".
      - Privado: integración con conceptos internos se realiza FUERA de la web (entorno local/RAG).
  - CITAR siempre que el hallazgo no sea conocimiento común; indicar fecha/versión.
  - Evitar citas literales >25 palabras.

SCOPE-IN:
  - Diseño de módulos de guía reutilizables (skills) y divulgación progresiva (<500 líneas + recursos).
  - Heurísticas de autoactivación (keywords, intent-regex, path globs, content patterns).
  - Hooks pre y post (formateo, build, hints, umbrales, auto-resolver).
  - Planning mode con aprobación previa (bloqueo de edición/ejecución).
  - Guardrails de BD/seguridad (p. ej., Prisma) y políticas (OPA).
  - PM2 para procesos backend y observabilidad.
  - KPIs: adherencia a guía, zero-errors-left-behind, mean-fix-latency, tokens/op, MTTD/MTTR.

SCOPE-OUT:
  - Cualquier mención, vínculo o inferencia sobre nombres/siglas propietarios del ecosistema privado.

METHOD:

- Buscar en dominios oficiales (docs, repos, blogs técnicos de fabricantes).
- Resumir patrones, límites, anti-patrones y trade-offs con fechas.
- Matriz comparativa entre "prácticas públicas" vs. "requisitos genéricos de un ecosistema privado" (sin siglas).
- Proponer contratos/plantillas neutrales (SKILL.md, rules.json, hooks API) y protocolo A/B de KPIs.

DELIVERABLES:
- 01_sota_public.md — Estado del arte (skills/hooks/plan/guardrails/PM2/KPIs) con fuentes y fechas.
- 02_patterns_limits.md — Patrones/anti-patrones y límites (token/latencia/ruido, lock-in, DX).
- 03_specs_neutral/
  - skill-rules.schema.json, skill-rules.example.json
  - SKILL.template.md (≤400 líneas + resources)
  - hooks.api.md (eventos, params, errores, casos de prueba)
  - plan-mode.spec.md (gates, permisos, rollback)
  - guardrails.db.spec.md (criterios block/suggest para Prisma/SQL)
- 04_kpis_experiments.md — KPIs, fórmulas, dataset piloto, umbrales PASS/NO-GO.
- 05_risks_roadmap.md — riesgos, mitigaciones, plan en olas.

QUALITY BAR:
- Citas (≤5 clave por sección) + fechas/versions.
- Contratos validados (JSON Schema) y ejemplos casi ejecutables.
- Checklist anti-alucinación pasado: sin siglas privadas en los outputs públicos.

---

4) Banco de consultas públicas (listas y “limpias”)

Usa estas plantillas sin tus siglas:

- Skills + divulgación progresiva

  “modular coding guidelines skills progressive disclosure site:docs.* OR site:github.com”

- IDE hooks / pre-prompt / post-response

  “IDE hooks user prompt submit stop event best practices code actions”

- Planning mode / gated execution

  “planning mode in IDE or agentic coding gated execution approval workflow”

- Guardrails DB (Prisma/SQL)

  “prisma query safety best practices guardrails where select principle”

- PM2

  “pm2 ecosystem config logs restart monit best practices”

- OPA / políticas

  “open policy agent best practices CI CD code quality gates”

- Telemetría/KPIs

  “engineering productivity kpis code quality adherence build error rate mean time to repair”

(Ajusta con dominios oficiales cuando sea posible.)

---

5) Utilidades de saneo (para tu CLI/agent)

Pseudocódigo TypeScript (envoltorio para cualquier motor de búsqueda):

```
const PRIVATE_TOKENS = [/\\bCLOOP\\b/i, /\\bBMCC\\b/i, /\\bS-Framework\\b/i, /\\bMemTech\\b/i, /\\bZen Hub Agent\\b/i, /\\bClaimTrace\\b/i];

export function sanitizeQuery(q: string): string {
  let s = q;
  for (const rx of PRIVATE_TOKENS) s = s.replaceAll(rx, ""); // elimina tokens privados
  return s.trim().replace(/\s{2,}/g, " ");
}

export function assertSafeQuery(q: string) {
  for (const rx of PRIVATE_TOKENS) {
    if (rx.test(q)) throw new Error("Unsafe query: contains private token");
  }
}

export function safeSearch(q: string, allowDomains: string[]) {
  const clean = sanitizeQuery(q);
  assertSafeQuery(clean);
```

```
  // construir consulta con filtros de dominio:
  // e.g., clean + " site:docs.anthropic.com OR site:pm2.keymetrics.io OR site:prisma.io OR site:openpolicyagent.org"
  return clean;
}
```

Checklist pre-publicación (por sección):

- ¿Se citan las afirmaciones no triviales?
- ¿No hay siglas privadas?
- ¿No hay inferencias sobre conceptos internos?
- ¿Fechas/versions claras?
- ¿Contratos neutrales y validados?

---

6) Flujo operativo (two-lane)

1. Carril público: ejecuta el prompt SAFE → produce 01–05 con citas.
2. Carril privado (local/RAG): toma 01–05 y mapea al glosario puente para integrarlo con tus conceptos (CLOOP/BMCC/S-Framework) sin navegar.
3. Validación: KPIs A/B (adherencia, zero-errors, latencia de fix).
4. Gate final: revisión anti-alucinación + ClaimTrace/ADR.

¿Quieres que convierta esto en un starter folder con el prompt SAFE, el glosario local, el schema JSON y el wrapper de “safeSearch” ya listos para tu CLI?

¡Listo! Aquí tienes un prompt de deep-research (listo para pegar en tu agente investigador) para construir una base teórica y técnica robusta que te permita implementar las mejoras tipo “Skills + Hooks + Plan Mode + Guardrails + PM2 + KPIs” dentro de tu ecosistema (CLOOP + CLI + MemTech/MCP Memoria + Zen Hub Agent + BMCC/S-Framework). Incluye objetivos, fuentes, metodología, entregables, KPIs y criterios de calidad.

DeepResearch Prompt — Implementación de Skills/Hooks/PlanMode/Guardrails en ECO-S (CLOOP+MemTech+ZenHub+BMCC)

ROLE: >
  Actúa como Investigador Técnico Senior + Arquitecto de Plataformas de IA.
  Dominios: ingeniería de software a escala, IDE/agents, agentic coding, MLOps/AIOps,
  gobernanza epistémica (BMCC/S-Framework), control de calidad, seguridad,
dev-experience.

MISSION: >
  Producir una base teórica y técnica exhaustiva, con evidencia y comparativos, para
  integrar un sistema tipo "Anthropic Skills + Hooks + Plan Mode" al ecosistema ECO-S
  (CLOOP + CLI + MemTech (MCP de memoria) + Zen Hub Agent + BMCC/S-Framework)
sin perder
  las ventajas actuales (gobernanza, on-prem, auditoría, PM2, ClaimTrace/OPA).

CONTEXT (RESUMEN): |
  - ECO-S existente:
    * CLOOP: método Clarify→Layout→Operate→Observe→Reflect con gates.
    * CLI CLOOP: comandos y hardening (build checks, verify-go).
    * MemTech (MCP Memoria): L1–L3, anti-drift, snapshots.
    * Zen Hub Agent: orquestación de agentes/roles con políticas.
    * BMCC + S-Framework: métricas epistémicas, sorpresa→γ, S-qual, ClaimTrace/OPA.
    * PM2: procesos backend, logs, restart/health.
  - Target conceptual a integrar:
    * Skills modulares + divulgación progresiva (<500 líneas + resources).
    * Hooks pre/post (auto-activación, formateo, build, hints/guardrails).
    * Plan Mode (planificación separada y aprobada antes de ejecución).
    * Guardrails por dominio (p. ej., base de datos).
    * KPIs de adherencia a skills, “zero-errors-left-behind”, mean-fix-latency.

OBJECTIVES:
  - O1: Mapear principios/arquitecturas de "skills + hooks + plan mode" y derivar patrones
replicables.
  - O2: Comparar, punto a punto, con ECO-S: quién cubre qué, brechas, superposiciones,
sinergias.
  - O3: Diseñar blueprint de integración (pre-invoke SkillRouter, StopHook, Plan Mode duro,
guardrails).
  - O4: Definir estándares de evidencia y KPIs operativos (skill-adherence, error-rate,
tokens/op, MTTD/MTTR).
  - O5: Entregar plantillas y contratos (SKILL.md, skill-rules.json, hooks API,
slash-commands, OPA/ClaimTrace).

SCOPE (IN/OUT):
  IN:
    - Diseño de Skill Bundles (core ≤400-500 líneas + resources + scripts adjuntos).
    - Heurísticas de auto-activación: keywords, intent-regex, globs de path, patrones de
contenido.
    - Hooks pre-invoque/post-respuesta: Prettier, tsc por repos tocados, hints/guardrails,
umbrales.
    - Plan Mode equivalente en ECO-S: bloqueos hasta PASS con C-LOOP Plan + ADR.

- Guardrails “bloqueantes” (DB/Prisma) y “no bloqueantes” (error-handling reminder).
- PM2 en flujo agentic (logs/restart/monit) como scripts adjuntos a skills.
- Telemetría/KPIs: adherencia a skill, zero-errors, mean-fix-latency, drift/γ.

OUT:
- Implementación de producto final; sólo artefactos de diseño, specs, ejemplos y pruebas piloto.

KEY QUESTIONS:
- KQ1: ¿Qué patrones/contratos mínimos definen un Skill Bundle efectivo (core+resources+scripts)?
- KQ2: ¿Cómo modelar auto-activación robusta (recall/precision) sin ruido excesivo? ¿Qué señales combinar?
- KQ3: ¿Qué pipeline post-respuesta maximiza “zero-errors-left-behind” con costo/latencia razonables?
- KQ4: ¿Cómo mapear Plan Mode ↔ C-LOOP para bloquear edición hasta aprobación verificable (ADR/ClaimTrace)?
- KQ5: ¿Qué guardrails deben ser “block” vs “suggest”? (DB, seguridad, auditoría, privacidad).
- KQ6: ¿Cómo se integran KPIs de skills con BMCC/S-Framework (sorpresa→γ, adherencia→S-qual)?
- KQ7: ¿Riesgos y mitigaciones? (lock-in, consumo de tokens, falsos positivos, latencia, DX).

DELIVERABLES:
- D1: State-of-the-Art Review (10–15p): principios, límites, trade-offs, referencias primarias.
- D2: Matriz comparativa “Skills/ClaudeCode” vs ECO-S por dimensión (auto-activación, hooks, plan, guardrails, KPIs, DX).
- D3: Blueprint de integración (diagrama + narrativa): pre-invoke router, stop-hook, plan-gate, guardrails, PM2, MemTech snapshots.
- D4: Especificaciones:
  * skill-rules.schema.json (JSON Schema) y ejemplo skill-rules.json (3-5 reglas).
  * SKILL.md (plantilla ≤400 líneas) + resources/* + scripts adjuntos (nombres/contratos).
  * Hooks API (TypeScript) con eventos/params/errores, pseudocódigo y casos de prueba.
  * Plan Mode Spec + gates (C-LOOP + ADR/ClaimTrace + permisos).
  * Guardrails DB (criterios block/suggest) y OPA/ClaimTrace mappings.
- D5: KPIs & Experimentos:
  * Definición de KPIs, dataset de pruebas, protocolo A/B, umbrales PASS/NO-GO.
- D6: Risk Register + Mitigaciones + Plan de adopción en 3 olas (rápida, escalamiento, endurecimiento).

METHODOLOGY:
- Phase 1 — Scoping & Sources:
  * Identifica documentación primaria (docs oficiales de skills/hooks/plan-mode, repos ejemplos),
  IDE/agents, PM2, Prisma/DB guardrails, OPA/ClaimTrace, métricas BMCC/S-Framework.

    * Incluye papers útiles (predictive processing, Bayesian surprise, EWC/SI, active inference)
      pero sólo como sostén conceptual (no dogma).
- Phase 2 — Evidence Review:
    * Resumen crítico (con citas y fechas); limitación: no exceder 25 palabras por cita literal.
    * Señala controvertidos y vacíos.
- Phase 3 — Comparative Analysis:
    * Tabla de capacidades y gaps con pesos (impacto x riesgo x esfuerzo).
- Phase 4 — Design & Contracts:
    * Proponer contratos (JSON/TS) y ejemplos concretos (mínimos viables).
- Phase 5 — KPIs & Experiments:
    * Diseñar protocolo A/B: medir adherencia y error-rate antes/después.
- Phase 6 — Roadmap & Risks:
    * Plan en olas (semana 1–2, mes 1, trimestre), con gates de GO/NO-GO.

SOURCES POLICY:
- Prioriza fuentes primarias y documentación oficial (Anthropic skills/Claude Code docs, repos oficiales).
- Complementa con PM2, Prisma/DB-safety, OPA/Reglas, métricas de ingeniería de software.
- Incluye fecha de publicación/actualización y versión; preferencia: ≤18 meses.
- Español/Inglés; si un tema no existe en ES, traduce hallazgos clave.

EVIDENCE STANDARDS:
- Citas con enlace y fecha (máx. 5 “citas de soporte” críticas por sección).
- Snippets literales ≤ 25 palabras; el resto, paráfrasis fiel.
- Señala supuestos/limitaciones y qué sería falsable o dependiente de versión del producto.

OUTPUT FORMATS:
- D1–D6 en Markdown + anexo con JSON/TS.
- Diagramas en Mermaid (o texto claro si no hay renderer).
- Entrega una carpeta “research-pack/” con:
    * 01_sota.md
    * 02_comparativa.md
    * 03_blueprint.md
    * 04_specs/
        - skill-rules.schema.json
        - skill-rules.example.json
        - SKILL.template.md
        - hooks.api.md (con casos de uso)
        - plan-mode.spec.md
        - guardrails.db.spec.md
    * 05_kpis_experimentos.md
    * 06_riesgos_y_roadmap.md

QUALITY BAR (DoD):
- Q1: Cobertura de todas las KQ con citas primarias y fechas.

- Q2: Matriz comparativa con veredictos claros por dimensión (quién lo hace mejor y por qué).
- Q3: Contratos ejecutables o casi-ejecutables (validados con JSON Schema).
- Q4: KPIs con definiciones formales, fórmulas, límites y ejemplos.
- Q5: Riesgos con probabilidad/impacto/mitigación y triggers operativos.

KPIs SUGERIDOS:
- skill_activation_recall / precision
- skill_adherence_rate (% de respuestas que cumplen la guía activa)
- zero_errors_left_behind (% respuestas sin TS errors tras StopHook)
- mean_fix_latency (s)
- tokens_per_operation / latency_ms
- drift_incidents (MemTech L1–L3) y Δγ (BMCC) correlacionado con adherencia

CONSTRAINTS:
- Evitar metáforas/jerga de videojuegos; terminología neutral (modular/orquestación/pipelines).
- Compatibilidad con on-prem y auditoría completa (ClaimTrace/OPA).
- Minimizar lock-in: abstraer contratos para que funcionen con múltiples LLMs/IDEs.

RISKS TO WATCH:
- Falsos positivos/negativos en auto-activación (ruido vs. recall).
- Consumo de tokens/latencia por demasiados recursos.
- “Skill rot” (desalineación entre guías y código real).
- Dependencia de hooking IDE/entorno; plan B: CLI hooks.
- Seguridad (scripts adjuntos a skills): firmar/verificar y sandbox.

CHECKLIST DE ENTREGA:
- [ ] D1 SOTA con 10–15p, ≤5 citas “clave” por sección, fechas y versiones.
- [ ] D2 Matriz comparativa con semáforos y justificación breve citada.
- [ ] D3 Blueprint (Mermaid + narrativa) con flujos pre/post.
- [ ] D4 Especificaciones (schema JSON, plantilla SKILL, hooks API, plan/guardrails).
- [ ] D5 KPIs + protocolo A/B + dataset de prueba propuesto.
- [ ] D6 Riesgos

¡Vamos! Hice la investigación “oficial” de Anthropic Skills y el ecosistema alrededor de Claude Code, y luego lo comparé—punto por punto—con tu CLOOP + CLI + MemTech (MCP memoria) + Zen Hub Agent. Al final te dejo acciones concretas para cerrar brechas sin perder tus ventajas.

Qué son “Skills” (resumen verificable)

- Módulos reutilizables (carpetas con SKILL.md, recursos y scripts) que Claude activa automáticamente cuando detecta que son pertinentes; están disponibles en Claude.ai, Claude Code, API y Agent SDK para Pro/Max/Team/Enterprise.

- Buenas prácticas: mantener SKILL.md < 500 líneas y usar divulgación progresiva hacia archivos de recursos (para eficiencia y foco contextual).
- Hooks de Claude Code: eventos como UserPromptSubmit (antes de que el modelo lea tu prompt) y Stop (post-respuesta) permiten inyectar recordatorios, verificar o post-formatear/build. (Hay ejemplos comunitarios y reporte de issues reales).
- Plan Mode: modo que separa planificación de ejecución (no edita ni corre nada hasta tu aprobación).
- Repos públicos con skills de ejemplo y documentación/overview de Claude Code.

Comparativa por capacidades (quién hace qué mejor)

Dimensión	Anthropic (Skills + Claude Code)	Tu stack (CLOOP + CLI + MemTech + Zen Hub)	Juicio / Acción
Auto-activación por contexto	Auto-invoca skills “relevantes” sin selección manual; soportado por hooks y heurísticas internas.	Tienes router/gating por contexto, pero la inyección pre-prompt no siempre es sistemática.	Empata/Mejorable en tu lado: añade pre-invoke SkillRouter (keywords/regex/globs/contenido) para inyectar “🎯 Skill Activation Check”.
Estructura de guías	SKILL.md < 500 líneas + recursos; patrón oficial de divulgación progresiva.	Guías extensas (BEST_PRACTICES/ADR).	Mejor Anthropic aquí: migra tus guías a bundles cortos + resources; ganarás en tokens/latencia y adherencia.
Hooks IDE/terminal	Soporta UserPromptSubmit/ Stop; ecosistema de	Tienes “post-build” y hardening en CLI, pero no un ciclo	Integra StopHook completo: prettier → tsc por repos

	ejemplos; issues conocidos documentados.	hook estandarizado pre/post-respuesta.	tocados → hints/guardrails → auto-resolver si ≥N errores.
Planificación segura	Plan Mode separa análisis de ejecución hasta aprobación.	Tu C-LOOP/ADR ya exige plan y gates, pero disperso entre comandos.	Paridad (con ventaja UX para Anthropic): crea un alias /plan que haga tu "C-LOOP Plan" + bloqueos de edición hasta PASS.
Guardrails por dominio	Skills pueden actuar como guardrails (p.ej., DB verification) y bloquear. (Patrón promovido en guías/best-practices).	ClaimTrace/OPA fuertes; buen gobierno epistémico.	Ventaja tuya en gobierno global; adopta guardrails de skill para checks locales (p.ej., Prisma/queries).
Portabilidad y despliegue	Skills corren en Claude.ai, Claude Code, API y Agent SDK; empresas ya los usan (Box, Canva, Rakuten).	Tu ecosistema es propio (Zen Hub, MCPs, MemTech) y portable en tus pipelines.	Ventaja Anthropic en alcance; tu ventaja: control total on-prem y auditoría estricta. Mantén ambos caminos.
Telemetría & KPIs	Docs públicas enfatizan prácticas, no KPIs de adherencia explícitos.	BMCC/S-qual con métricas (adherencia, latencia de fix, drift, sorpresa).	Ventaja tuya: mantén KPIs y añade skill-adherence% como métrica primaria.
Gestión de procesos/logs	Claude Code integra terminal; PM2 no es parte nativa (prensa sugiere "code	Tienes PM2/scripts/monitoreo; reinicios/lectura	Ventaja tuya: conserva PM2 + expón comandos al agente; documenta

	execution tool beta” para ciertos flujos).	de logs automatizados.	como utility scripts dentro de skills.
Distribución/versionado	Skills versionables, repos públicos y tooling oficial/SDK.	Tú ya versionas ADR/skills internos en git + MemTech snapshots.	Paridad: añade snapshots de Skill Bundles en MemTech L1 para anti-drift.
Multi-agente	Subagentes/“agents & tools” en la plataforma; amplia adopción ecosistema.	Zen Hub Agent/ACE/ADR/RUME bien definidos con roles/gates.	Ventaja tuya en orquestación gobernada; suma skills como contratos de cada agente.

Cosas que

Anthropic hace y tú no (aún)

- Activación automática end-to-end de skills (sin pedirlo) visible en el flujo de trabajo del modelo; es parte de la experiencia oficial.
- Plantillas y repos de ejemplo listos para clonar (ahorra tiempo inicial).
- Plan Mode con UX muy pulido desde el terminal/IDE.

Cosas que

tú haces y Anthropic no cubre de fábrica

- Gobernanza epistémica (BMCC/S-qual/ClaimTrace/OPA) con métricas operativas accionables.
- Operación/observabilidad PM2 + health-checks + restart + lectura de logs en caliente.
- Rutas de despliegue on-prem y auditoría completa de decisiones/modelo (tus ADR/CLAIMTRACE).

Acciones concretas (sin romper tu identidad)

1. Refactor de guías → Skill Bundles
    - Reescribe tus BEST_PRACTICES en módulos ≤400 líneas + resources y publícalos como bundles internos. (Alineado a la guía oficial <500).
2. Pre-invoke SkillRouter + StopHook estándar
    - Pre-invoke: inyecta "🎯 Skill Activation Check" (keywords/regex/globs).
    - Post: prettier → tsc por repos tocados → hints/guardrail → auto-resolver si ≥5 errores. (Patrón coherente con hooks CC y tu hardening).
3. /plan "duro"
    - Emula Plan Mode: bloquear edición/ejecución hasta PASS de C-LOOP Plan firmado (PR-gate).
4. Guardrail DB como skill bloqueante
    - Skill "database-verification" que bloquea queries inseguras y exige lectura de recursos (Prisma). (Patrón reconocido).
5. KPIs de skills en BMCC
    - Añade skill-activation recall/precision, adherence%, mean-fix-latency, tokens/op al tablero de S-qual.
6. Scripts adjuntos
    - Documenta scripts clave (PM2, auth-route, seed/reset) dentro de cada skill para que el agente use herramientas existentes (en vez de "generar otra vez").

Si quieres, te lo empaqueto como "CLOOP Skill Pack" (plantillas SKILL.md, resources/, skill-rules.json, hooks pre/post, ejemplos PM2) listo para pegar en tu monorepo.

Totalmente de acuerdo: ese post es "tu sistema, pero llevado al máximo" con otra terminología. Aprovechemos la intersección para cerrar brechas sin perder lo que ya te hace fuerte (CLOOP, MemTech, BMCC, Zen MCP, ClaimTrace).

Comparativa rápida (lado-a-lado)

Capacidad	Tu stack (Zen MCP + MemTech + BMCC + CLOOP)	Post de Claude Code	Oportunidad concreta
Patrones/Guías	Meta-prompts, ADR, BEST_PRAC, MemTech L1–L3	Skills modulares (<500 líneas + recursos)	Convertir BEST_PRAC en Skill Bundles (archivo corto + recursos) con niveles guideline/guardrail.
Auto-activación	Gating por contexto + C-LOOP; no siempre “pre-invoke” automático	Hook UserPromptSubmit que sugiere skills por keywords/regex/rutas	Añadir SkillRouter pre-invoke (keywords, intent regex, globs, patrones de contenido) que inyecte “🎯 Skill Activation Check”.
QA post-respuesta	verify-go.sh, hardening, disciplina; checks manuales	Hook Stop: Prettier, build por repo, umbrales, “error-hints”	Formalizar StopHook: prettier → tsc por repos cambiados → hints → auto-resolver si ≥5 errores.
Dev-docs por tarea	ADR + C-LOOP; docs extensas	Tríada plan.md / context.md / tasks.md en /dev/active/	Estandarizar tríada reproducible y sincronizarla con MemTech (snapshot L1) antes de compaction.

Gestión procesos/logs	Logs por servicio; acceso a mano	PM2 con logs por servicio, restart y monitoreo	Incorporar PM2 + comandos del agente para leer pm2 logs <svc>/restart; health-checks.
Agentes especializados	ACE/ADR/Hub; RUME en diseño	Sub-agentes muy específicos (tester, fixer, planner)	“Productizar” 6 agentes críticos: build-error-resolver, code-architecture-reviewer, auth-route-tester, frontend-error-fixer, plan-architect, plan-reviewer.
Slash-commands	CLI y comandos C-LOOP; no homogéneos	/dev-docs, /build-and-fix, /code-review	Normalizar 4 slash-cmds canónicos con prompts largos pegados.
Guardrails	OPA, ClaimTrace, políticas; fuerte en auditoría	Guardrails por skill (p.ej., DB verification bloqueante)	Añadir guardrail “database-verification” que bloquee ediciones inseguras (Prisma) y obligue lectura de recursos.
Eficiencia de tokens	Documentos grandes; compaction con MemTech	Skills cortos + recursos progresivos	Reescribir guías a ≤400 líneas + “resources/” enlazadas; medir ahorro de tokens y latencia.

Telemetría/éxito	BMCC, Surprise Metrics, S-qual	Métricas implícitas	Exponer KPIs de adherencia: %skill-adherence, error-rate post-hook, tiempo-a-fix, “zero-errors-left-behind”.
Scripts adjuntos	Scripts útiles; no siempre referenciados	Cada skill referencia scripts concretos	Enlazar scripts canónicos desde cada Skill Bundle (p.ej., scripts/test-auth-route.js).
Planificación	C-LOOP Plan → Operate → Observe → Reflect	planning mode + plan-architect	Integrar strategic-plan-architect como plantilla oficial (produce la tríada + riesgos + métricas).

Top 7 mejoras (priorizadas por impacto/tiempo)

1. Skill Bundles + SkillRouter (pre-invoke)
    - Entregable: tools/skill-rules.json (+ schema), skills/<domain>/core.md (≤400 líneas) + resources/*.md.
    - GATE: 90% de prompts con nota “🎯 Skill Activation Check” cuando corresponde; false-positive rate <10%.
2. StopHook completo (post-respuesta)
    - Pipeline: prettier → tsc por repos tocados → hints → auto-resolver si ≥5 errores.

- GATE: “zero-errors-left-behind” = 100% en PRs de prueba durante 1 semana.

3. Tríada de dev-docs + MemTech snapshot
    - dev/active/<task>/{plan,context,tasks}.md + snapshot L1 al cerrar ciclo.
    - GATE: Re-inicio de sesión sin pérdida de hilo en ≥3 tareas largas consecutivas.
4. PM2 en microservicios
    - ecosystem.config.cjs + comandos del agente para logs/restart/monit.
    - GATE: MTTD de errores backend < 2 min gracias a pm2 logs.
5. Guardrail DB bloqueante
    - Skill “database-verification” con enforcement block (find/update sin where → STOP).
    - GATE: 0 incidentes por query insegura en 2 sprints.
6. Slash-commands estandarizados
    - /dev-docs, /dev-docs-update, /build-and-fix, /code-review.
    - GATE: 80% de acciones repetibles invocadas vía slash-cmd.
7. KPIs BMCC para adherencia a skills
    - Métricas: skill-activation recall/precision, guideline-adherence %, mean-fix-latency, tokens/op.
    - GATE: Dashboard con 4 KPIs y umbrales de alerta.

Plantillas mínimas (listas para pegar)

A)

tools/skill-rules.json

(semilla)

```
{
  "backend-dev-guidelines": {
    "type": "guideline",
    "enforcement": "suggest",
    "priority": "high",
    "promptTriggers": {
      "keywords": ["backend","controller","service","endpoint","API"],
      "intentPatterns": ["(create|add|migrate).*?(route|endpoint|controller)","(how to|best
practice).*?(backend|API)"]
    },
    "fileTriggers": {
      "pathPatterns": ["services/**/src/**/*.ts","backend/**/*.ts"],
      "contentPatterns": ["router\\.","export\\s+class\\s+.*Controller","@prisma/client"]
    },
    "resources": [
      "docs/skills/backend/01-routes.md",
      "docs/skills/backend/02-controllers.md",
      "docs/skills/backend/03-repositories.md"
    ]
  },
  "database-verification": {
    "type": "guardrail",
    "enforcement": "block",
    "priority": "critical",
    "promptTriggers": { "keywords": ["prisma","schema","migration"] },
    "fileTriggers": {
      "pathPatterns": ["**/*.prisma","services/**/src/repositories/**/*.ts"],
      "contentPatterns": ["prisma\\.","findMany\\(","update\\("]
    },
    "resources": ["docs/skills/db/guardrails.md"]
  },
  "frontend-dev-guidelines": {
    "type": "guideline",
    "enforcement": "suggest",
    "priority": "high",
    "promptTriggers": { "keywords": ["react","mui","tanstack","router","query"] },
    "fileTriggers": { "pathPatterns": ["frontend/**/{src,app}/**/*.{ts,tsx}"] },
    "resources": [
      "docs/skills/frontend/01-components.md",
      "docs/skills/frontend/02-hooks.md",
      "docs/skills/frontend/03-query-router.md"
    ]
  }
}
```

B) Nota de inyección (pre-invoke)

🎯 SKILL ACTIVATION CHECK
- Use skill: <name> (enforcement/priority)
  · resource: docs/skills/<domain>/<file>.md

C) Tríada de dev-docs (front-matter mínimo)

plan.md

Plan — <tarea>
Objetivo: …
Alcance: …
Fases: …
Riesgos/Mitigación: …
Métricas de éxito: …
Last Updated: 2025-10-29

context.md

Context — <tarea>
Archivos clave: …
Decisiones (ADR refs): …
Dependencias: …
Próximos pasos: …
Last Updated: 2025-10-29

tasks.md

Tasks — <tarea>
- [ ] Paso 1 …
- [ ] Paso 2 …
- [ ] Validación/Tests …
Last Updated: 2025-10-29

Cómo encaja con tu

S-Framework / BMCC

- Skill adherence y drift: si el SkillRouter activa una guía y la respuesta no cumple (detectado por code-architecture-reviewer), incrementa sorpresa epistémica → sube γ puntual para reforzar la representación correcta (memoria L1/L2) y exigir plan/revisión adicional.

- S-qual: añade submétricas: %adherencia a skill, #errores post-StopHook, latencia de corrección, tokens/op.

- C-LOOP: Clarify (plan) → Layout (skills/resources) → Operate (edits) → Observe (StopHook/PM2) → Reflect (BMCC KPIs + ClaimTrace).

Plan de adopción express (48 h de trabajo, ejecutable por etapas)

1. Día 1 (mañana): crear tools/skill-rules.json + migrar 2 guías (backend/front) a core+resources.

2. Día 1 (tarde): habilitar pre-invoke SkillRouter en tu runner MCP; inyectar nota.

3. Día 2 (mañana): implementar StopHook (prettier + tsc + hints + umbral).

4. Día 2 (tarde): estandarizar tríada dev-docs + snapshot MemTech; añadir PM2 a 2 servicios.

Si quieres, te empaqueto estos artefactos como “CLOOP Action Pack” (carpetas + scripts + ejemplos de skills) para que los pegues directo en el monorepo.

Felipe, qué joya de post. Está ultra alineado con lo que vienes construyendo (CLOOP, MemTech, BMCC, Zen MCP). Para convertirlo en paquete ejecutable hoy en tu stack (Cursor/Zen-Hub/CLI), te dejo un Starter Pack minimal con hooks, skills auto-activables, PM2, dev-docs y slash-commands. Lo pegas en tu monorepo y corre.

CLOOP Action Pack (15-min drop-in)

Estructura sugerida

```
/tools/agent-hooks/
  src/
    index.ts
    hooks.userPromptSubmit.ts
    hooks.stop.ts
    detectors.ts
    build-check.ts
```

```
    format.ts
    error-hints.ts
  skill-rules.schema.json
  skill-rules.json
  tsconfig.json
  package.json
/scripts/pm2/ecosystem.config.cjs
/dev/active/            # se autogenera por comandos
/docs/CLAUDE.md         # (o) /docs/AGENT.md
/docs/PROJECT_KNOWLEDGE.md
/docs/TROUBLESHOOTING.md
/bin/cloop-dev.mjs
```

---

1) Reglas para auto-activar Skills

/tools/agent-hooks/skill-rules.schema.json

```
{
  "$schema":"http://json-schema.org/draft-07/schema#",
  "title":"SkillRules",
  "type":"object",
  "patternProperties":{
    "^[a-z0-9-]+$":{
      "type":"object",
      "required":["type","enforcement","priority","promptTriggers","fileTriggers"],
      "properties":{
        "type":{"enum":["domain","guideline","guardrail"]},
        "enforcement":{"enum":["suggest","require","block"]},
        "priority":{"enum":["low","normal","high","critical"]},
        "promptTriggers":{
          "type":"object",
          "properties":{
            "keywords":{"type":"array","items":{"type":"string"}},
            "intentPatterns":{"type":"array","items":{"type":"string"}}
          }
        },
        "fileTriggers":{
          "type":"object",
          "properties":{
            "pathPatterns":{"type":"array","items":{"type":"string"}},
            "contentPatterns":{"type":"array","items":{"type":"string"}}
          }
        },
        "resources":{"type":"array","items":{"type":"string"}}
      }
    }
```

```
  },
  "additionalProperties":false
}
```

/tools/agent-hooks/skill-rules.json (ejemplo inicial)

```
{
  "backend-dev-guidelines": {
    "type": "guideline",
    "enforcement": "suggest",
    "priority": "high",
    "promptTriggers": {
      "keywords": ["backend","controller","service","endpoint","API"],
      "intentPatterns": ["(create|add|migrate).*?(route|endpoint|controller)","(how to|best
practice).*?(backend|API)"]
    },
    "fileTriggers": {
      "pathPatterns": ["backend/**/*.ts","services/**/src/**/*.ts"],
      "contentPatterns": ["router\\.","export\\s+class\\s+.*Controller","@prisma/client"]
    },
    "resources": [
      "docs/skills/backend/01-routes.md",
      "docs/skills/backend/02-controllers.md",
      "docs/skills/backend/03-repositories.md"
    ]
  },
  "database-verification": {
    "type": "guardrail",
    "enforcement": "block",
    "priority": "critical",
    "promptTriggers": { "keywords": ["prisma","schema","migration"] },
    "fileTriggers": {
      "pathPatterns": ["**/*.prisma","backend/**/repositories/**/*.ts"],
      "contentPatterns": ["prisma\\.","findMany\\(","update\\("]
    },
    "resources": ["docs/skills/db/guardrails.md"]
  },
  "frontend-dev-guidelines": {
    "type": "guideline",
    "enforcement": "suggest",
    "priority": "high",
    "promptTriggers": { "keywords": ["react","mui","tanstack","router","query"] },
    "fileTriggers": { "pathPatterns": ["frontend/**/{src,app}/**/*.{ts,tsx}"] },
    "resources": [
      "docs/skills/frontend/01-components.md",
      "docs/skills/frontend/02-hooks.md",
      "docs/skills/frontend/03-query-router.md"
    ]
```

```
  }
}
```

---

2) Hooks pre y post (proveedor-agnóstico)

/tools/agent-hooks/src/index.ts

```
import { loadRules, matchRulesFor } from "./detectors";
import { runPrettier } from "./format";
import { buildCheck } from "./build-check";
import { emitErrorHints } from "./error-hints";

export type EditLog = { file:string; repo:string; ts:number }[];
export type PreHookResult = { injectedNote?:string; activated:string[] };

export async function userPromptSubmitHook(input:{
  prompt:string; openFiles:string[]; cwd:string;
}):Promise<PreHookResult>{
  const rules = await loadRules();
  const { activated, note } = matchRulesFor({ rules, prompt:input.prompt, files:input.openFiles
});
  return { injectedNote: note, activated };
}

export async function stopHook(input:{
  editLog:EditLog; reposChanged:Set<string>; cwd:string;
}){
  await runPrettier(input.editLog.map(e=>e.file));
  const results = await buildCheck([...input.reposChanged], input.cwd);
  await emitErrorHints({ editLog:input.editLog, build:results });
  return results; // útil para que tu CLI decida si dispara un agente “auto-error-resolver”
}
```

/tools/agent-hooks/src/detectors.ts

```
import fs from "node:fs/promises";
import path from "node:path";

type Rule = any; // simplificado

export async function loadRules(){
  const p = path.resolve(process.cwd(), "tools/agent-hooks/skill-rules.json");
  return JSON.parse(await fs.readFile(p,"utf8")) as Record<string,Rule>;
}

export function
matchRulesFor({rules,prompt,files}:{rules:Record<string,Rule>,prompt:string,files:string[]}){
```

```
  const activated:string[] = [];
  const lower = prompt.toLowerCase();
  const noteLines:string[] = ["🎯 SKILL ACTIVATION CHECK"];
  for(const [name,rule] of Object.entries(rules)){
    const kw = (rule.promptTriggers?.keywords??[]).some((k:string)=>
lower.includes(k.toLowerCase()));
    const re = (rule.promptTriggers?.intentPatterns??[]).some((p:string)=> new
RegExp(p,"i").test(prompt));
    const pathHit = (rule.fileTriggers?.pathPatterns??[]).some((glob:string)=> files.some(f=>
minimatchLike(f,glob)));
    if(kw || re || pathHit){
      activated.push(name);
      noteLines.push(`- Use skill: ${name} (${rule.enforcement}/${rule.priority})`);
      for(const r of (rule.resources||[])) noteLines.push(`  · resource: ${r}`);
    }
  }
  const note = activated.length ? noteLines.join("\n") : undefined;
  return { activated, note };
}

// glob simple tipo **/*.ts
function minimatchLike(file:string, pattern:string){
  const esc = pattern.replace(/\./g,"\\.").replace(/\*\*/g,".+").replace(/\*/g,"[^/]+");
  return new RegExp("^"+esc+"$").test(file);
}
```

/tools/agent-hooks/src/format.ts

```
import { execa } from "execa";
export async function runPrettier(files:string[]){
  if(!files.length) return;
  await execa("npx", ["prettier","--write", ...files], { stdio:"inherit" });
}
```

/tools/agent-hooks/src/build-check.ts

```
import { execa } from "execa";
export type BuildResult = { repo:string; ok:boolean; errors:number; out:string };

export async function buildCheck(repos:string[], cwd:string){
  const out:BuildResult[] = [];
  for(const repo of repos){
    const cmd = ["-C", repo, "pnpm","-s","-w","tsc","-p","tsconfig.json","--noEmit"];
    try{
      const { stdout } = await execa("git", cmd, { cwd });
      out.push({ repo, ok:true, errors:0, out:stdout });
    }catch(err:any){
      const text = (err.stdout||"") + "\n" + (err.stderr||"");
```

```
      const n = (text.match(/error TS\d+/g)||[]).length;
      out.push({ repo, ok:false, errors:n, out:text });
    }
  }
  return out;
}
```

/tools/agent-hooks/src/error-hints.ts

```
export async function emitErrorHints({editLog, build}:{editLog:any[], build:any[]}){
  const risky = editLog.filter(e=> /controller|service|repository|api|router|\.prisma/.test(e.file));
  if(!risky.length) return;
  const lines = [
    "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━",
    "📋 ERROR HANDLING SELF-CHECK",
    "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━",
    `⚠️  Backend Changes Detected: ${risky.length} file(s)`,
    "❓ Did you add Sentry.captureException() in catch blocks?",
    "❓ Are Prisma operations wrapped in error handling?",
    "💡 Controllers should extend BaseController; repos → patrón de repositorio."
  ];
  console.log(lines.join("\n"));
}
```

/tools/agent-hooks/package.json

```
{
  "name":"agent-hooks",
  "type":"module",
  "private":true,
  "dependencies":{ "execa":"^9.3.0" },
  "devDependencies":{ "typescript":"^5.6.3" }
}
```

> Cómo usarlo en tu flujo: tu CLI (o MCP) llama a userPromptSubmitHook(...) antes de invocar el modelo (inyecta injectedNote al contexto) y a stopHook(...) después de cada respuesta, pasándole el edit-log acumulado y los repos tocados (tu runner ya los tiene porque sabe qué archivos editó el agente).

---

3) PM2 para microservicios (debug autónomo)

/scripts/pm2/ecosystem.config.cjs

```
module.exports = {
  apps: [
```

```
    { name:'auth',  cwd:'./services/auth',  script:'pnpm', args:'start', error_file:'logs/error.log', out_file:'logs/out.log' },
    { name:'email', cwd:'./services/email', script:'pnpm', args:'start', error_file:'logs/error.log', out_file:'logs/out.log' },
    { name:'forms', cwd:'./services/forms', script:'pnpm', args:'start', error_file:'logs/error.log', out_file:'logs/out.log' }
  ]
};
```

Comandos útiles:

```
pnpm pm2:start   # alias a: pm2 start scripts/pm2/ecosystem.config.cjs
pm2 logs email --lines 200
pm2 restart email
pm2 monit
```

En tu agente, expón herramientas para leer pm2 logs <svc> y tomar acciones (reinicio, health-check).

---

4) Dev-Docs reproducibles (evita “perder el hilo”)

/bin/cloop-dev.mjs

```
#!/usr/bin/env node
import fs from "node:fs/promises";
import path from "node:path";

const [cmd, name] = process.argv.slice(2);

async function ensureDir(p){ await fs.mkdir(p, { recursive:true }); }
async function now(){ return new Date().toISOString().replace(/\.\d+Z$/,"Z"); }

if(cmd === "start" && name){
  const root = path.resolve("dev/active", name);
  await ensureDir(root);
  await fs.writeFile(path.join(root, `${name}-plan.md`), `# Plan\n- [ ] ...\n\n_Last Updated: ${await now()}_\n`);
  await fs.writeFile(path.join(root, `${name}-context.md`), `# Context\nFiles:\n\n_Last Updated: ${await now()}_\n`);
  await fs.writeFile(path.join(root, `${name}-tasks.md`), `# Tasks\n- [ ] Task 1\n\n_Last Updated: ${await now()}_\n`);
  console.log("✅ Dev docs creados en", root);
  process.exit(0);
}
if(cmd === "update" && name){
  const root = path.resolve("dev/active", name);
```

```
  for (const f of [`${name}-plan.md`,`${name}-context.md`,`${name}-tasks.md`]){
    const p = path.join(root,f);
    let t = await fs.readFile(p,"utf8");
    t = t.replace(/_Last Updated:[^_]*_/g, `_Last Updated: ${await now()}_`);
    await fs.writeFile(p,t);
  }
  console.log("✍️  Timestamps actualizados");
  process.exit(0);
}
console.log("Uso: cloop-dev.mjs start <task-name> | update <task-name>");
process.exit(1);
```

Workflow

- /plan: generas plan con tu agente → cloop-dev start <tarea> → pega el plan y contexto → implementa por secciones → cloop-dev update <tarea> antes de compaction.

---

5) CLAUDE.md (o AGENT.md) mínimo y separación limpia

/docs/CLAUDE.md

```
# CLAUDE.md (raíz)

## Reglas universales (100 líneas máx)
- No merges con TypeScript errors.
- Usa skills: `frontend-dev-guidelines`, `backend-dev-guidelines`, `database-verification`.

## Repos específicos
- Ver `/frontend/docs/claude.md`, `/services/email/docs/claude.md`.

## Dev Docs
- Activa plan: `bin/cloop-dev.mjs start <tarea>`
- Mantén `/dev/active/<tarea>/*` actualizado.

## HERRAMIENTAS
- PM2: `pm2 logs <svc>`, `pm2 restart <svc>`
- Hooks: pre/post habilitados vía `agent-hooks`.
```

Y coloca best-practices en skills (archivos cortos + recursos modulares), no en CLAUDE.md.

---

6) Slash-commands (plantillas expandibles)

Configúralos en tu runner/CLI o como “snippets” en Cursor:

- /dev-docs → expande al prompt generador de plan (con campos: objetivos, fases, riesgos, métricas éxito).
- /dev-docs-update → llama script anterior y pide al modelo marcar tareas completadas y próximas acciones.
- /build-and-fix → dispara stopHook con auto-error-resolver si errors >= 5.
- /code-review → invoca tu agente code-architecture-reviewer (lista de checks + FAIL/OK por archivo).

---

7) Guardrails rápidos

- database-verification con enforcement: “block”:
    - Si detecta prisma/repo, obliga a leer docs/skills/db/guardrails.md.
    - Rechaza edición si falta where/select o si hay any en repositorios.
- error-handling recordatorio no bloqueante (ya incluido).
- build-checker siempre tras cada respuesta.

---

8) Integraciones con tu ecosistema

- Zen MCP / Agents Hub: monta agent-hooks como módulo utility. Los MCPs “Context7 / MemTech / ADR” llaman userPromptSubmitHook y stopHook.
- MemTech: persiste /dev/active/<tarea>/* y snapshots de skill-rules.json + hashes (evita drift).
- BMCC: registra métricas “zero-errors-left-behind”, “guideline-adherence%”, y “mean-fix-latency”.

---

9) Scripts adjuntos a skills (patrón clave)

En cada skill, referencia scripts ejecutables concretos (p.ej., scripts/test-auth-route.js) y cómo invocarlos. Claude/GPT no “improvisa”: usa la herramienta existente.

---

10) Próximos pasos (rápidos)

1. Copia los archivos arriba en tu monorepo y pnpm -C tools/agent-hooks i.
2. En tu runner (CLI/MCP), agrega:
    - pre-invoke → userPromptSubmitHook(...) y pega injectedNote al contexto.
    - post-invoke → stopHook(...) con editLog + reposChanged.
3. Mueve BEST_PRACTICES a /docs/skills/** (≤400 líneas cada uno, recursos aparte).
4. Añade ecosystem PM2 y alias pnpm pm2:start.
5. Define 3 slash-commands: /dev-docs, /dev-docs-update, /build-and-fix.

Si quieres, te dejo esto en un zip listo para descargar y con ejemplos de skills (frontend, backend, db-guardrails). ¿Lo preparo así mismo y lo subo?

Claude Code is a Beast – Tips from 6 Months of Hardcore Use

Quick pro-tip from a fellow lazy person: You can throw this book of a post into one of the many text-to-speech AI services like [*ElevenLabs Reader*](https://elevenlabs.io/text-reader) *or* [*Natural Reader*](https://www.naturalreaders.com/online/) *and have it read the post for you* :)

Disclaimer

I made a post about six months ago sharing my experience after a week of hardcore use with Claude Code. It's now been about six months of hardcore use, and I would like to share some more tips, tricks, and word vomit with you all. I may have went a little overboard here so strap in, grab a coffee, sit on the toilet or whatever it is you do when doom-scrolling reddit.

I want to start the post off with a disclaimer: all the content within this post is merely me sharing what setup is working best for me currently and should not be taken as gospel or the only correct way to do things. It's meant to hopefully inspire you to improve your setup and workflows with AI agentic coding. I'm just a guy, and this is just like, my opinion, man.

Also, I'm on the 20x Max plan, so your mileage may vary. And if you're looking for vibe-coding tips, you should look elsewhere. If you want the best out of CC, then you should be working together with it: planning, reviewing, iterating, exploring different approaches, etc.

Quick Overview

After 6 months of pushing Claude Code to its limits (solo rewriting 300k LOC), here's the system I built:

* Skills that actually auto-activate when needed

* Dev docs workflow that prevents Claude from losing the plot

* PM2 + hooks for zero-errors-left-behind

* Army of specialized agents for reviews, testing, and planning

Let's get into it.

Background

I'm a software engineer who has been working on production web apps for the last seven years or so. And I have fully embraced the wave of AI with open arms. I'm not too worried about AI taking my job anytime soon, as it is a tool that I use to leverage my capabilities. In doing so, I have been building MANY new features and coming up with all sorts of new proposal presentations put together with Claude and GPT-5 Thinking to integrate new AI systems into our production apps. Projects I would have never dreamt of having the time to even consider before integrating AI into my workflow. And with all that, I'm giving myself a good deal of job security and have become the AI guru at my job since everyone else is about a year or so behind on how they're integrating AI into their day-to-day.

With my newfound confidence, I proposed a pretty large redesign/refactor of one of our web apps used as an internal tool at work. This was a pretty rough college student-made project that was forked off another project developed by me as an intern (created about 7 years ago and forked 4 years ago). This may have been a bit overly ambitious of me since, to sell it to the stakeholders, I agreed to finish a top-down redesign of this fairly decent-sized project (\~100k LOC) in a matter of a few months...all by myself. I knew going in that I was going to have to put in extra hours to get this done, even with the help of CC. But deep down, I know it's going to be a hit, automating several manual processes and saving a lot of time for a lot of people at the company.

It's now six months later... yeah, I probably should not have agreed to this timeline. I have tested the limits of both Claude as well as my own sanity trying to get this thing done. I completely scrapped the old frontend, as everything was seriously outdated and I wanted to play with the latest and greatest. I'm talkin' React 16 JS → React 19 TypeScript, React Query v2 → TanStack Query v5, React Router v4 w/ hashrouter → TanStack Router w/ file-based routing, Material UI v4 → MUI v7, all with strict adherence to best practices. The project is now at \~300-400k LOC and my life expectancy \~5 years shorter. It's finally ready to put up for testing, and I am incredibly happy with how things have turned out.

This used to be a project with insurmountable tech debt, ZERO test coverage, HORRIBLE developer experience (testing things was an absolute nightmare), and all sorts of jank going on. I addressed all of those issues with decent test coverage, manageable tech debt, and implemented a command-line tool for generating test data as well as a dev mode to test different features on the frontend. During this time, I have gotten to know CC's abilities and what to expect out of it.

A Note on Quality and Consistency

I've noticed a recurring theme in forums and discussions - people experiencing frustration with usage limits and concerns about output quality declining over time. I want to be clear up front: I'm not here to dismiss those experiences or claim it's simply a matter of "doing it wrong." Everyone's use cases and contexts are different, and valid concerns deserve to be heard.

That said, I want to share what's been working for me. In my experience, CC's output has actually improved significantly over the last couple of months, and I believe that's largely due to the workflow I've been constantly refining. My hope is that if you take even a small bit of inspiration from my system and integrate it into your CC workflow, you'll give it a better chance at producing quality output that you're happy with.

Now, let's be real - there are absolutely times when Claude completely misses the mark and produces suboptimal code. This can happen for various reasons. First, AI models are stochastic, meaning you can get widely varying outputs from the same input. Sometimes the randomness just doesn't go your way, and you get an output that's legitimately poor quality through no fault of your own. Other times, it's about how the prompt is structured. There can be significant differences in outputs given slightly different wording because the model takes things quite literally. If you misword or phrase something ambiguously, it can lead to vastly inferior results.

Sometimes You Just Need to Step In

Look, AI is incredible, but it's not magic. There are certain problems where pattern recognition and human intuition just win. If you've spent 30 minutes watching Claude struggle with something that you could fix in 2 minutes, just fix it yourself. No shame in that. Think of it like teaching someone to ride a bike, sometimes you just need to steady the handlebars for a second before letting go again.

I've seen this especially with logic puzzles or problems that require real-world common sense. AI can brute-force a lot of things, but sometimes a human just "gets it" faster. Don't let stubbornness or some misguided sense of "but the AI should do everything" waste your time. Step in, fix the issue, and keep moving.

I've had my fair share of terrible prompting, which usually happens towards the end of the day where I'm getting lazy and I'm not putting that much effort into my prompts. And the results really show. So next time you are having these kinds of issues where you think the output is way worse these days because you think Anthropic shadow-nerfed Claude, I encourage you to take a step back and reflect on how you are prompting.

Re-prompt often. You can hit double-esc to bring up your previous prompts and select one to branch from. You'd be amazed how often you can get way better results armed with the knowledge of what you don't want when giving the same prompt. All that to say, there can be many reasons why the output quality seems to be worse, and it's good to self-reflect and consider what you can do to give it the best possible chance to get the output you want.

As some wise dude somewhere probably said, "Ask not what Claude can do for you, ask what context you can give to Claude" \~ Wise Dude

Alright, I'm going to step down from my soapbox now and get on to the good stuff.

My System

I've implemented a lot changes to my workflow as it relates to CC over the last 6 months, and the results have been pretty great, IMO.

Skills Auto-Activation System (Game Changer!)

This one deserves its own section because it completely transformed how I work with Claude Code.

The Problem

So Anthropic releases this Skills feature, and I'm thinking "this looks awesome!" The idea of having these portable, reusable guidelines that Claude can reference sounded perfect for maintaining consistency across my massive codebase. I spent a good chunk of time with Claude writing up comprehensive skills for frontend development, backend development, database operations, workflow management, etc. We're talking thousands of lines of best practices, patterns, and examples.

And then... nothing. Claude just wouldn't use them. I'd literally use the exact keywords from the skill descriptions. Nothing. I'd work on files that should trigger the skills. Nothing. It was incredibly frustrating because I could see the potential, but the skills just sat there like expensive decorations.

The "Aha!" Moment

That's when I had the idea of using **hooks**. If Claude won't automatically use skills, what if I built a system that MAKES it check for relevant skills before doing anything?

So I dove into Claude Code's hook system and built a multi-layered auto-activation architecture with TypeScript hooks. And it actually works!

How It Works

I created two main hooks:

1. UserPromptSubmit Hook (runs BEFORE Claude sees your message):

* Analyzes your prompt for keywords and intent patterns

* Checks which skills might be relevant

* Injects a formatted reminder into Claude's context

* Now when I ask "how does the layout system work?" Claude sees a big "🎯 SKILL ACTIVATION CHECK - Use project-catalog-developer skill" (project catalog is a large complex data grid based feature on my front end) before even reading my question

2. Stop Event Hook (runs AFTER Claude finishes responding):

* Analyzes which files were edited

* Checks for risky patterns (try-catch blocks, database operations, async functions)

* Displays a gentle self-check reminder

* "Did you add error handling? Are Prisma operations using the repository pattern?"

* Non-blocking, just keeps Claude aware without being annoying

skill-rules.json Configuration

I created a central configuration file that defines every skill with:

* **Keywords**: Explicit topic matches ("layout", "workflow", "database")

* **Intent patterns**: Regex to catch actions ("(create|add).*?(feature|route)")

* **File path triggers**: Activates based on what file you're editing

* **Content triggers**: Activates if file contains specific patterns (Prisma imports, controllers, etc.)

Example snippet:

```
  {
    "backend-dev-guidelines": {
      "type": "domain",
      "enforcement": "suggest",
      "priority": "high",
      "promptTriggers": {
        "keywords": ["backend", "controller", "service", "API", "endpoint"],
        "intentPatterns": [
          "(create|add).*?(route|endpoint|controller)",
```

```
          "(how to|best practice).*?(backend|API)"
        ]
      },
      "fileTriggers": {
        "pathPatterns": ["backend/src/**/*.ts"],
        "contentPatterns": ["router\\.", "export.*Controller"]
      }
    }
  }
```

The Results

Now when I work on backend code, Claude automatically:

1. Sees the skill suggestion before reading my prompt

2. Loads the relevant guidelines

3. Actually follows the patterns consistently

4. Self-checks at the end via gentle reminders

The difference is night and day. No more inconsistent code. No more "wait, Claude used the old pattern again." No more manually telling it to check the guidelines every single time.

Following Anthropic's Best Practices (The Hard Way)

After getting the auto-activation working, I dove deeper and found Anthropic's official best practices docs. Turns out I was doing it wrong because they recommend keeping the main SKILL.md file **under 500 lines** and using progressive disclosure with resource files.

Whoops. My frontend-dev-guidelines skill was 1,500+ lines. And I had a couple other skills over 1,000 lines. These monolithic files were defeating the whole purpose of skills (loading only what you need).

So I restructured everything:

* **frontend-dev-guidelines**: 398-line main file + 10 resource files

* **backend-dev-guidelines**: 304-line main file + 11 resource files

Now Claude loads the lightweight main file initially, and only pulls in detailed resource files when actually needed. Token efficiency improved 40-60% for most queries.

Skills I've Created

Here's my current skill lineup:

Guidelines & Best Practices:

* `backend-dev-guidelines` \- Routes → Controllers → Services → Repositories

* `frontend-dev-guidelines` \- React 19, MUI v7, TanStack Query/Router patterns

* `skill-developer` \- Meta-skill for creating more skills

Domain-Specific:

* `workflow-developer` \- Complex workflow engine patterns

* `notification-developer` \- Email/notification system

* `database-verification` \- Prevent column name errors (this one is a guardrail that actually blocks edits!)

* `project-catalog-developer` \- DataGrid layout system

All of these automatically activate based on what I'm working on. It's like having a senior dev who actually remembers all the patterns looking over Claude's shoulder.

Why This Matters

Before skills + hooks:

* Claude would use old patterns even though I documented new ones

* Had to manually tell Claude to check BEST_PRACTICES.md every time

* Inconsistent code across the 300k+ LOC codebase

* Spent too much time fixing Claude's "creative interpretations"

After skills + hooks:

* Consistent patterns automatically enforced

* Claude self-corrects before I even see the code

* Can trust that guidelines are being followed

* Way less time spent on reviews and fixes

If you're working on a large codebase with established patterns, I cannot recommend this system enough. The initial setup took a couple of days to get right, but it's paid for itself ten times over.

CLAUDE.md and Documentation Evolution

In a post I wrote 6 months ago, I had a section about rules being your best friend, which I still stand by. But my CLAUDE.md file was quickly getting out of hand and was trying to do too much. I also had this massive BEST_PRACTICES.md file (1,400+ lines) that Claude would sometimes read and sometimes completely ignore.

So I took an afternoon with Claude to consolidate and reorganize everything into a new system. Here's what changed:

What Moved to Skills

Previously, BEST_PRACTICES.md contained:

* TypeScript standards

* React patterns (hooks, components, suspense)

* Backend API patterns (routes, controllers, services)

* Error handling (Sentry integration)

* Database patterns (Prisma usage)

* Testing guidelines

* Performance optimization

All of that is now in skills with the auto-activation hook ensuring Claude actually uses them. No more hoping Claude remembers to check BEST_PRACTICES.md.

What Stayed in CLAUDE.md

Now CLAUDE.md is laser-focused on **project-specific info** (only \~200 lines):

* Quick commands (`pnpm pm2:start`, `pnpm build`, etc.)

* Service-specific configuration

* Task management workflow (dev docs system)

* Testing authenticated routes

* Workflow dry-run mode

* Browser tools configuration

The New Structure

Root CLAUDE.md (100 lines)

├── Critical universal rules

├── Points to repo-specific claude.md files

└── References skills for detailed guidelines

Each Repo's claude.md (50-100 lines)

├── Quick Start section pointing to:

│ ├── PROJECT_KNOWLEDGE.md - Architecture & integration

│ ├── TROUBLESHOOTING.md - Common issues

│ └── Auto-generated API docs

└── Repo-specific quirks and commands

The magic: Skills handle all the "how to write code" guidelines, and CLAUDE.md handles "how this specific project works." Separation of concerns for the win.

Dev Docs System

This system, out of everything (besides skills), I think has made the most impact on the results I'm getting out of CC. Claude is like an extremely confident junior dev with extreme amnesia, losing track of what they're doing easily. This system is aimed at solving those shortcomings.

The dev docs section from my CLAUDE.md:

Starting Large Tasks

When exiting plan mode with an accepted plan: 1.**Create Task Directory**:

mkdir -p ~/git/project/dev/active/[task-name]/

2.**Create Documents**:

- `[task-name]-plan.md` - The accepted plan

- `[task-name]-context.md` - Key files, decisions

- `[task-name]-tasks.md` - Checklist of work

3.**Update Regularly**: Mark tasks complete immediately

Continuing Tasks

- Check `/dev/active/` for existing tasks

- Read all three files before proceeding

- Update "Last Updated" timestamps

These are documents that always get created for every feature or large task. Before using this system, I had many times when I all of a sudden realized that Claude had lost the plot and we were no longer implementing what we had planned out 30 minutes earlier because we went off on some tangent for whatever reason.

My Planning Process

My process starts with planning. **Planning is king**. If you aren't at a minimum using planning mode before asking Claude to implement something, you're gonna have a bad time, mmm'kay. You wouldn't have a builder come to your house and start slapping on an addition without having him draw things up first.

When I start planning a feature, I put it into planning mode, even though I will eventually have Claude write the plan down in a markdown file. I'm not sure putting it into planning mode necessary, but to me, it feels like planning mode gets better results doing the research on your codebase and getting all the correct context to be able to put together a plan.

I created a `strategic-plan-architect` subagent that's basically a planning beast. It:

* Gathers context efficiently

* Analyzes project structure

* Creates comprehensive structured plans with executive summary, phases, tasks, risks, success metrics, timelines

* Generates three files automatically: plan, context, and tasks checklist

But I find it really annoying that you can't see the agent's output, and even more annoying is if you say no to the plan, it just kills the agent instead of continuing to plan. So I also created a custom slash command (`/dev-docs`) with the same prompt to use on the main CC instance.

Once Claude spits out that beautiful plan, I take time to review it thoroughly. **This step is really important.** Take time to understand it, and you'd be surprised at how often you catch silly mistakes or Claude misunderstanding a very vital part of the request or task.

More often than not, I'll be at 15% context left or less after exiting plan mode. But that's okay because we're going to put everything we need to start fresh into our dev docs. Claude usually likes to just jump in guns blazing, so I immediately slap the ESC key to interrupt and run my `/create-dev-docs` slash command. The command takes the approved plan and creates all three files, sometimes doing a bit more research to fill in gaps if there's enough context left.

And once I'm done with that, I'm pretty much set to have Claude fully implement the feature without getting lost or losing track of what it was doing, even through an auto-compaction. I just make sure to remind Claude every once in a while to update the tasks as well as the context file with any relevant context. And once I'm running low on context in the current session, I just run my slash command `/update-dev-docs`. Claude will note any relevant context (with next steps) as well as mark any completed tasks or add new tasks before I compact the conversation. And all I need to say is "continue" in the new session.

During implementation, depending on the size of the feature or task, I will specifically tell Claude to only implement one or two sections at a time. That way, I'm getting the chance to go in and review the code in between each set of tasks. And periodically, I have a subagent also reviewing the changes so I can catch big mistakes early on. If you aren't having Claude review its own code, then I highly recommend it because it saved me a lot of headaches catching critical errors, missing implementations, inconsistent code, and security flaws.

PM2 Process Management (Backend Debugging Game Changer)

This one's a relatively recent addition, but it's made debugging backend issues so much easier.

The Problem

My project has seven backend microservices running simultaneously. The issue was that Claude didn't have access to view the logs while services were running. I couldn't just ask "what's going wrong with the email service?" - Claude couldn't see the logs without me manually copying and pasting them into chat.

The Intermediate Solution

For a while, I had each service write its output to a timestamped log file using a `devLog` script. This worked... okay. Claude could read the log files, but it was clunky. Logs weren't real-time, services wouldn't auto-restart on crashes, and managing everything was a pain.

The Real Solution: PM2

Then I discovered PM2, and it was a game changer. I configured all my backend services to run via PM2 with a single command: `pnpm pm2:start`

What this gives me:

* Each service runs as a managed process with its own log file

* **Claude can easily read individual service logs in real-time**

* Automatic restarts on crashes

* Real-time monitoring with `pm2 logs`

* Memory/CPU monitoring with `pm2 monit`

* Easy service management (`pm2 restart email`, `pm2 stop all`, etc.)

PM2 Configuration:

```
    // ecosystem.config.jsmodule.exports = {
      apps: [
        {
          name: 'form-service',
          script: 'npm',
          args: 'start',
          cwd: './form',
          error_file: './form/logs/error.log',
          out_file: './form/logs/out.log',
        },
    // ... 6 more services
      ]
    };
```

Before PM2:

```
    Me: "The email service is throwing errors"
    Me: [Manually finds and copies logs]
    Me: [Pastes into chat]
    Claude: "Let me analyze this..."
```

The debugging workflow now:

```
    Me: "The email service is throwing errors"
    Claude: [Runs] pm2 logs email --lines 200
```

Claude: [Reads the logs] "I see the issue - database connection timeout..."

Claude: [Runs] pm2 restart email

Claude: "Restarted the service, monitoring for errors..."

Night and day difference. Claude can autonomously debug issues now without me being a human log-fetching service.

One caveat: Hot reload doesn't work with PM2, so I still run the frontend separately with `pnpm dev`. But for backend services that don't need hot reload as often, PM2 is incredible.

Hooks System (#NoMessLeftBehind)

The project I'm working on is multi-root and has about eight different repos in the root project directory. One for the frontend and seven microservices and utilities for the backend. I'm constantly bouncing around making changes in a couple of repos at a time depending on the feature.

And one thing that would annoy me to no end is when Claude forgets to run the build command in whatever repo it's editing to catch errors. And it will just leave a dozen or so TypeScript errors without me catching it. Then a couple of hours later I see Claude running a build script like a good boy and I see the output: "There are several TypeScript errors, but they are unrelated, so we're all good here!"

No, we are not good, Claude.

Hook #1: File Edit Tracker

First, I created a **post-tool-use hook** that runs after every Edit/Write/MultiEdit operation. It logs:

* Which files were edited

* What repo they belong to

* Timestamps

Initially, I made it run builds immediately after each edit, but that was stupidly inefficient. Claude makes edits that break things all the time before quickly fixing them.

Hook #2: Build Checker

Then I added a **Stop hook** that runs when Claude finishes responding. It:

1. Reads the edit logs to find which repos were modified

2. Runs build scripts on each affected repo

3. Checks for TypeScript errors

4. If < 5 errors: Shows them to Claude

5. If ≥ 5 errors: Recommends launching auto-error-resolver agent

6. Logs everything for debugging

Since implementing this system, I've not had a single instance where Claude has left errors in the code for me to find later. The hook catches them immediately, and Claude fixes them before moving on.

Hook #3: Prettier Formatter

This one's simple but effective. After Claude finishes responding, automatically format all edited files with Prettier using the appropriate `.prettierrc` config for that repo.

No more going into to manually edit a file just to have prettier run and produce 20 changes because Claude decided to leave off trailing commas last week when we created that file.

Hook #4: Error Handling Reminder

This is the gentle philosophy hook I mentioned earlier:

* Analyzes edited files after Claude finishes

* Detects risky patterns (try-catch, async operations, database calls, controllers)

* Shows a gentle reminder if risky code was written

* Claude self-assesses whether error handling is needed

* No blocking, no friction, just awareness

Example output:

💡 Backend Best Practice:

- All errors should be captured to Sentry

- Controllers should extend BaseController

The Complete Hook Pipeline

Here's what happens on every Claude response now:

Claude finishes responding

↓

Hook 1: Prettier formatter runs → All edited files auto-formatted

↓

Hook 2: Build checker runs → TypeScript errors caught immediately

↓

Hook 3: Error reminder runs → Gentle self-check for error handling

↓

If errors found → Claude sees them and fixes

↓

If too many errors → Auto-error-resolver agent recommended

↓

Result: Clean, formatted, error-free code

And the UserPromptSubmit hook ensures Claude loads relevant skills BEFORE even starting work.

No mess left behind. It's beautiful.

Scripts Attached to Skills

One really cool pattern I picked up from Anthropic's official skill examples on GitHub: **attach utility scripts to skills**.

For example, my `backend-dev-guidelines` skill has a section about testing authenticated routes. Instead of just explaining how authentication works, the skill references an actual script:

    ### Testing Authenticated Routes

    Use the provided test-auth-route.js script:

    node scripts/test-auth-route.js http://localhost:3002/api/endpoint

The script handles all the complex authentication steps for you:

1. Gets a refresh token from Keycloak

2. Signs the token with JWT secret

3. Creates cookie header

4. Makes authenticated request

When Claude needs to test a route, it knows exactly what script to use and how to use it. No more "let me create a test script" and reinventing the wheel every time.

I'm planning to expand this pattern - attach more utility scripts to relevant skills so Claude has ready-to-use tools instead of generating them from scratch.

Tools and Other Things

SuperWhisper on Mac

Voice-to-text for prompting when my hands are tired from typing. Works surprisingly well, and Claude understands my rambling voice-to-text surprisingly well.

Memory MCP

I use this less over time now that skills handle most of the "remembering patterns" work. But it's still useful for tracking project-specific decisions and architectural choices that don't belong in skills.

BetterTouchTool

* Relative URL copy from Cursor (for sharing code references)

    * I have VSCode open to more easily find the files I’m looking for and I can double tap CAPS-LOCK, then BTT inputs the shortcut to copy relative URL, transforms the clipboard

contents by prepending an ‘@’ symbol, focuses the terminal, and then pastes the file path. All in one.

* Double-tap hotkeys to quickly focus apps (CMD+CMD = Claude Code, OPT+OPT = Browser)

* Custom gestures for common actions

Honestly, the time savings on just not fumbling between apps is worth the BTT purchase alone.

Scripts for Everything

If there's any annoying tedious task, chances are there's a script for that:

* Command-line tool to generate mock test data. Before using Claude code, it was extremely annoying to generate mock data because I would have to make a submission to a form that had about a 120 questions Just to generate one single test submission.

* Authentication testing scripts (get tokens, test routes)

* Database resetting and seeding

* Schema diff checker before migrations

* Automated backup and restore for dev database

Pro tip: When Claude helps you write a useful script, immediately document it in CLAUDE.md or attach it to a relevant skill. Future you will thank past you.

Documentation (Still Important, But Evolved)

I think next to planning, documentation is almost just as important. I document everything as I go in addition to the dev docs that are created for each task or feature. From system architecture to data flow diagrams to actual developer docs and APIs, just to name a few.

But here's what changed: Documentation now works WITH skills, not instead of them.

Skills contain: Reusable patterns, best practices, how-to guides **Documentation contains:** System architecture, data flows, API references, integration points

For example:

* "How to create a controller" → **backend-dev-guidelines skill**

* "How our workflow engine works" → **Architecture documentation**

* "How to write React components" → **frontend-dev-guidelines skill**

* "How notifications flow through the system" → **Data flow diagram + notification skill**

I still have a LOT of docs (850+ markdown files), but now they're laser-focused on project-specific architecture rather than repeating general best practices that are better served by skills.

You don't necessarily have to go that crazy, but I highly recommend setting up multiple levels of documentation. Ones for broad architectural overview of specific services, wherein you'll include paths to other documentation that goes into more specifics of different parts of the architecture. It will make a major difference on Claude's ability to easily navigate your codebase.

Prompt Tips

When you're writing out your prompt, you should try to be as specific as possible about what you are wanting as a result. Once again, you wouldn't ask a builder to come out and build you a new bathroom without at least discussing plans, right?

"You're absolutely right! Shag carpet probably is not the best idea to have in a bathroom."

Sometimes you might not know the specifics, and that's okay. If you don't ask questions, tell Claude to research and come back with several potential solutions. You could even use a specialized subagent or use any other AI chat interface to do your research. The world is your oyster. I promise you this will pay dividends because you will be able to look at the plan that Claude has produced and have a better idea if it's good, bad, or needs adjustments. Otherwise, you're just flying blind, pure vibe-coding. Then you're gonna end up in a situation where you don't even know what context to include because you don't know what files are related to the thing you're trying to fix.

Try not to lead in your prompts if you want honest, unbiased feedback. If you're unsure about something Claude did, ask about it in a neutral way instead of saying, "Is this good or bad?" Claude tends to tell you what it thinks you want to hear, so leading questions can skew the response. It's better to just describe the situation and ask for thoughts or alternatives. That way, you'll get a more balanced answer.

Agents, Hooks, and Slash Commands (The Holy Trinity)

Agents

I've built a small army of specialized agents:

Quality Control:

* `code-architecture-reviewer` \- Reviews code for best practices adherence

* `build-error-resolver` \- Systematically fixes TypeScript errors

* `refactor-planner` \- Creates comprehensive refactoring plans

Testing & Debugging:

* `auth-route-tester` \- Tests backend routes with authentication

* `auth-route-debugger` \- Debugs 401/403 errors and route issues

* `frontend-error-fixer` \- Diagnoses and fixes frontend errors

Planning & Strategy:

* `strategic-plan-architect` \- Creates detailed implementation plans

* `plan-reviewer` \- Reviews plans before implementation

* `documentation-architect` \- Creates/updates documentation

Specialized:

* `frontend-ux-designer` \- Fixes styling and UX issues

* `web-research-specialist` \- Researches issues along with many other things on the web

* `reactour-walkthrough-designer` \- Creates UI tours

The key with agents is to give them **very specific roles** and clear instructions on what to return. I learned this the hard way after creating agents that would go off and do who-knows-what and come back with "I fixed it!" without telling me what they fixed.

Hooks (Covered Above)

The hook system is honestly what ties everything together. Without hooks:

* Skills sit unused

* Errors slip through

* Code is inconsistently formatted

* No automatic quality checks

With hooks:

* Skills auto-activate

* Zero errors left behind

* Automatic formatting

* Quality awareness built-in

Slash Commands

I have quite a few custom slash commands, but these are the ones I use most:

Planning & Docs:

* `/dev-docs` \- Create comprehensive strategic plan

* `/dev-docs-update` \- Update dev docs before compaction

* `/create-dev-docs` \- Convert approved plan to dev doc files

Quality & Review:

* `/code-review` \- Architectural code review

* `/build-and-fix` \- Run builds and fix all errors

Testing:

* `/route-research-for-testing` \- Find affected routes and launch tests

* `/test-route` \- Test specific authenticated routes

The beauty of slash commands is they expand into full prompts, so you can pack a ton of context and instructions into a simple command. Way better than typing out the same instructions every time.

Conclusion

After six months of hardcore use, here's what I've learned:

The Essentials:

1. **Plan everything** \- Use planning mode or strategic-plan-architect

2. **Skills + Hooks** \- Auto-activation is the only way skills actually work reliably

3. **Dev docs system** \- Prevents Claude from losing the plot

4. **Code reviews** \- Have Claude review its own work

5. **PM2 for backend** \- Makes debugging actually bearable

The Nice-to-Haves:

* Specialized agents for common tasks

* Slash commands for repeated workflows

* Comprehensive documentation

* Utility scripts attached to skills

* Memory MCP for decisions

And that's about all I can think of for now. Like I said, I'm just some guy, and I would love to hear tips and tricks from everybody else, as well as any criticisms. Because I'm always up

for improving upon my workflow. I honestly just wanted to share what's working for me with other people since I don't really have anybody else to share this with IRL (my team is very small, and they are all very slow getting on the AI train).

If you made it this far, thanks for taking the time to read. If you have questions about any of this stuff or want more details on implementation, happy to share. The hooks and skills system especially took some trial and error to get right, but now that it's working, I can't imagine going back.

TL;DR: Built an auto-activation system for Claude Code skills using TypeScript hooks, created a dev docs workflow to prevent context loss, and implemented PM2 + automated error checking. Result: Solo rewrote 300k LOC in 6 months with consistent quality.